RediSwap: MEV Redistribution Mechanism for CFMMs

Mengqian Zhang
Yale University
mengqian.cs@gmail.com

Sen Yang
Yale University
sen.yang@yale.edu

Fan Zhang
Yale University
f.zhang@yale.edu

Abstract

Automated Market Makers (AMMs) are essential to decentralized finance, offering continuous liquidity and enabling intermediary-free trading on blockchains. However, participants in AMMs are vulnerable to Maximal Extractable Value (MEV) exploitation. Users face threats such as front-running, back-running, and sandwich attacks, while liquidity providers (LPs) incur loss-versus-rebalancing (LVR).

In this paper, we introduce RediSwap, a novel AMM designed to capture MEV at the application level and refund it among users and liquidity providers. At its core, RediSwap features an MEV-redistribution mechanism that manages MEV opportunities arising from the rebalancing arbitrage and pending transactions within the AMM pool. Our mechanism sells these MEV opportunities to arbitrageurs via an auction-style process based on their reported price beliefs. The mechanism determines transaction sequencing and inserts MEV transactions on behalf of winning arbitrageurs. Payments from arbitrageurs are refunded to users and LPs, mitigating MEV on AMMs.

Theoretically, we prove that our mechanism is dominant-strategy incentive compatible, individually rational, and Sybil-proof. Empirically, we compare RediSwap with existing solutions by replaying historical AMM trades. Our results suggest that RediSwap can achieve better execution than UniswapX in 89% of trades and reduce liquidity providers' loss to under 0.5% of the original LVR in most cases. By safeguarding the interests of users and liquidity providers, our work offers a new framework for MEV redistribution in decentralized markets.

Keywords: Decentralized Finance, MEV Redistribution, Mechanism Design

1 Introduction

Automated Market Makers (AMMs) [1] have become the leading design for facilitating trades on decentralized exchanges (DEXs). A key feature of AMMs is their use of liquidity pools, which are composed of tokens contributed by liquidity providers (LPs) and accumulated from past trades. This structure enables users to trade directly with the pool, eliminating the need for order matching as seen in the traditional order book system used by centralized exchanges (CEXs).

Despite various benefits, two types of participants in AMMs — users and LPs — suffer from the phenomenon known as Maximal/Miner Extractable Value (MEV) [2]. MEV refers to the profit that intermediaries, such as searchers, builders, and proposers, can extract during the block production. Because trades in DEXs are publicly visible in the mempool before they are confirmed, those monitoring the mempool (e.g., searchers) can profit by front-running, back-running, or sandwiching user trades [3, 4, 5]. As a result, user trades execute at a worse price. For liquidity providers, the primary risk comes from Loss-Versus-Rebalancing (LVR) [6], which quantifies the cost they incur when arbitrageurs rebalance the AMM pools in response to price movements at external markets. The existence of LVR makes it challenging for liquidity providers to earn sustainable profits, even with swap fees.

One promising direction to mitigate users' loss is *refunding* them partial MEV extracted from their trades. MEV-Share [7] and MEV Blocker [8] are primary examples used in practice. However, these refunding mechanisms are rather limited, mainly because they operate near the end of the MEV supply chain, after the trades have been exploited by various intermediaries, such as searchers/arbitrageurs and builders. Moreover, some (e.g., application-level) information is lost as transactions pass through the supply chain. One consequence is unfair redistribution. For instance, in MEV Blocker, the majority of refunds for CoWSwap [9] orders were erroneously delivered to solvers rather than users [10].

On the other hand, UniswapX [11] and CoWSwap are two notable solutions at the application level, an *upstream* stage of the supply chain. In both systems, users submit intents specifying their desired outcomes, and a market of solvers compete to search for the best settlement. These schemes are expected to provide better execution because solvers can aggregate liquidity from various sources and access more resources (information, capital, and sophisticated execution strategies) than users. However, empirical evidence (as detailed in Appendix A) suggests that solvers may not exhibit the expected level of sophistication, emphasizing the need for simpler mechanism designs.

Several parallel works have been proposed to mitigate LVR and compensate liquidity providers. A widely studied approach is to auction off exclusive rights earlier before block generation in exchange for compensation to LPs [12, 13]. A major concern with these *ex-ante* auctions is that arbitrageurs must bid based on their estimation of future profits, which makes it challenging to engage and discourages risk-averse players. Additionally, other LVR mitigation ideas have been explored, such as reducing the inter-block time and involving dynamic fees [14, 15]. However, like all previous MEV mitigation solutions, these approaches focus solely on protecting the interests of one group (LPs in this case), while do not take the other group of participants into account.

The limitations of existing solutions motivate us to explore designs that can capture MEV at the application level, refund MEV among users and LPs, and ensure ease of participation for solvers (arbitrageurs). Particularly, we tackle the research question through the lens of mechanism design.

1.1 This Work

In this paper, we propose an MEV-redistribution Constant Function Market Maker (CFMM) called RediSwap. RediSwap addresses the above limitations: the refund mechanism takes both users and LPs into consideration; the arbitrageurs only need to take simple actions to participate. Looking ahead, our empirical evaluation suggests that RediSwap achieves better execution results than existing systems while mitigating LVR.

RediSwap has two main components: a CFMM and an MEV-redistribution mechanism that manages arbitrage opportunities[1] within the CFMM pool. We focus on the CFMM with a risky asset and a numéraire. The CFMM follows the standard design, except that users send trades to the MEV-redistribution mechanism, which sequences the trades and uses the CFMM to execute them; the CFMM does not accept trades directly from users.

The overall workflow of RediSwap is as follows: users privately send transactions to the MEV-redistribution mechanism, e.g., via an RPC channel. Arbitrageurs interested in the potential arbitrage opportunity provide the MEV-redistribution mechanism with their beliefs regarding the external market price of the risky asset. The MEV-redistribution mechanism is essentially an *ex-post* auction to sell MEV opportunities to arbitrageurs, specifying three key rules: (1) a bundle generation rule, which constructs a list of transactions (i.e., a bundle) with optimal arbitrage profit based on the inputs from users and arbitrageurs; (2) a payment rule, which decides arbitrageurs' payments, capturing a portion of the MEV from the bundle; and (3) a refund rule, which rebates the captured MEV among users and liquidity providers.

While RediSwap relies on arbitrageurs (as with UniswapX and CoWSwap), arbitrageurs only need to take simple actions, i.e., to provide their price beliefs and capital. That is, RediSwap internalizes both the complexity of computing MEV strategies and the competition among arbitrageurs within the mechanism. Moreover, RediSwap has full control over transaction ordering, which enables transparent MEV capturing and can enforce fair redistribution. Unlike MEV-Share or MEV Blocker, RediSwap does not expose transaction information to arbitrageurs.

1.2 Our Contributions

We present the first dominant-strategy incentive compatible (DSIC), individually rational (IR), and Sybil-proof MEV-redistribution mechanism for CFMMs.

In section 3, we formalize the problem of designing an MEV-redistribution mechanism — a composition of a bundle generation rule, payment rule, and refund rule — and specify the desired properties. Intuitively, we aim for a mechanism that is DSIC for arbitrageurs, meaning that truthfully reporting beliefs regarding the risk asset price is a dominant strategy, and IR, ensuring that truthful arbitrageurs always receive nonnegative utility. Moreover, in the open and decentralized blockchain environment, arbitrageurs might pose as users and

[1] We focus on non-atomic arbitrage, which capitalizes on price discrepancies between on-chain DEXs and exchanges in another venue (i.e., CEXs like Binance or DEXs on other blockchains). Non-atomic arbitrage is one of the dominant forms of MEV. Empirical studies indicate that over a quarter of the trading volume on Ethereum's biggest five DEXs is likely attributed to non-atomic arbitrage [16], and since the Ethereum Merge [17], the total profits from CEX-DEX arbitrage have reached $131.77M [18].

mount *Sybil attacks* by submitting "fake" transactions to steal users' refunds. Thus, we also require the mechanism to be *Sybil-proof*, in the sense that creating Sybil transactions will not reduce any user's utility. Such a mechanism would make it simple for arbitrageurs to participate and not require much sophistication.

Section 4 introduces an auction-style mechanism that sells MEV opportunities arising from pending transactions and rebalancing arbitrage to arbitrageurs. We begin by defining the value of a pool state and a transaction to reflect their potential MEV value to an arbitrary arbitrageur. Using the reports provided by arbitrageurs, the mechanism generates "bids" on their behalf according to these valuation functions. Winning arbitrageurs are awarded the MEV opportunities, and the mechanism *automatically* adds the necessary MEV transactions (rebalancing or "sandwiching") for winners so that they can realize the MEV value. A primary challenge in constructing the overall MEV bundle is that transaction execution in CFMMs is order-sensitive, and so is the MEV value it creates. We address this by constructing a special "sandwich" for each transaction — one that always starts from the initial state and eventually returns to the initial state again — ensuring that MEV extraction for different opportunities remains independent. Ultimately, each winner pays the second-highest bid for the MEV opportunity; these payments are then refunded to liquidity providers (for rebalancing arbitrage, corresponding to LVR) and to users (for sandwich attacks).

Theoretically, we prove that the mechanism is DSIC (Theorem 1), IR (Theorem 2), and Sybil-proof (Theorem 3). Note that our definition of Sybil-proofness focuses on users' utility (rather than the arbitrageurs'), reflecting the mechanism's core goal: protecting users (and LPs) from MEV exploitation. One may notice that this definition differs from the conventional manner, which emphasizes whether the attacker can improve their own utility via Sybil identities. In many classic Sybil settings, resources of interest are usually fixed (e.g., items in auctions [19], bandwidth in BitTorrent [20], winning positions in voting [21]), making it a zero-sum situation — if Sybil attackers gain more, others necessarily lose. Hence, stopping attackers from benefiting is equivalent to protecting others. By contrast, in the MEV context, each new transaction, including the Sybil one, can potentially create *additional* MEV value, thus increasing the total MEV "resources" to be redistributed. Consequently, we directly safeguard users' utility: In essence, the Sybil-proof theorem states that submitting Sybil transactions does not lower any user's utility. This, however, may enhance the arbitrageur's own utility, and further incentivize her to misreport (Example 2). To address this concern, we show that if an arbitrageur holds some capital to launch Sybil attacks, there exists a Sybil strategy such that using this strategy and *truthfully reporting* is a Nash equilibrium (Theorem 4). As a result, users and LPs receive their desired utility under our mechanism (i.e., the utility when arbitrageurs report truthfully), despite potential Sybil behavior.

In section 5, we evaluate our mechanism by replaying historical AMM trades from September 1st, 2023, to August 31st, 2024. To simulate arbitrageurs' beliefs in external prices, we use prices in Binance (a centralized exchange) to build price distribution. By comparing the execution prices of the same order on UniswapX or CoWSwap with RediSwap, we show that our mechanism can achieve better execution prices than UniswapX in 89% of cases, and comparable execution prices to CoWSwap. Our evaluation also shows that RediSwap effectively reduces LVR for the two Uniswap v2 liquidity pools (WETH-USDC and

WETH-USDT) we experiment with, and the efficiency strengthens as more arbitrageurs participate. For example, WETH-USDC liquidity providers incur less than 0.5% of the LVR they would face without RediSwap in most cases.

Further directions to improve the mechanism are discussed in section 6.

2 Related Work

MEV mitigation through AMM design. Several works focus on composing networks of AMMs to achieve better settlement and minimize arbitrage and slippage [22, 23, 24]. Instead of executing transactions sequentially, some designs [9, 25] process transactions in batches at a uniform clearing price, eliminating the risk of cyclic arbitrage and sandwich attacks within a block, though not all forms of manipulation are fully addressed [26]. Some studies have also explored the idea of integrating batch trading with CFMMs [27, 28]. Orthogonally, Ferreira and Parkes [29] initiate the study of verifiable sequencing rules and propose a concrete greedy sequencing rule, which structurally inhibits the feasibility of sandwich attacks. Chan, Wu, and Shi [30] initiate the mechanism design approach/philosophy for mitigating MEV and study a model where relevant strategic players (user or miner) aim to obtain *risk-free profit*, whereas we focus on non-atomic arbitrage. AnimaguSwap [31] presents an AMM design to achieve data-independent ordering of user transactions at the application level. V0LVER [32] presents an AMM against LVR and MEV based on an encrypted transaction mempool.

Other MEV mitigation solutions. Mitigating the negative effects of MEV is a research focus in both academia and industry, with various studies conducted from different perspectives. A commonly used practice for MEV mitigation is using private channels [8, 33], where user transactions are sent directly to block builders, bypassing the public mempool and thus preventing MEV attacks like front-running and sandwich attacks. Another approach focuses on ensuring time-based order fairness [34, 35, 36, 37]. Miners or validators adhering to time-based order fairness must order transactions based on the time they receive the transaction, preventing MEV caused by ex-post order manipulation. A different method to achieve fair ordering involves users first committing to their transactions and revealing them only after the order has been determined. Thus, the transaction order is determined independently of the content. We refer readers to an SoK paper [38] for a comprehensive understanding of various MEV mitigation approaches.

Sybil-proofness in mechanism design in blockchain. The permissionless nature of blockchain makes it very easy for participants to create multiple identities, thus launching Sybil attacks. Since the pioneered work by Roughgarden on transaction fee mechanism design [39] and also for practical consideration, Sybil-proofness has become one of the most desired properties for mechanism design in blockchain. Notable examples include the rich transaction fee mechanism design literature [39, 40, 41, 42, 43], voting mechanism [44], and the recent mechanism in ZK-Rollup prover markets [45]. Mazorra and Penna [46] consider a similar MEV rebates problem in MEV-share combinatorial order flow auctions. They discuss the Shapley value-based mechanism and show that in their setting, any symmetric, efficient, and Strong monotonic rebate mechanism is weak against Sybil strategies.

3 Model

3.1 The Basic Model

This section describes the basic CFMM pool and all involved players, including liquidity providers, users, and arbitrageurs.

CFMM Pool. We consider a CFMM pool with a risky asset $\mathcal{X}$ (e.g., ETH) and a numéraire asset $\mathcal{Y}$ (e.g., USDC). The pool state $s = (x, y)$ is specified by the current reserves of tokens and should satisfy a constant function $F(x, y)$. The slope at a state $(x, y) \in F$ is the spot price or marginal exchange rate $\left|\frac{\partial F/\partial x}{\partial F/\partial y}\right|$. We do not prescribe a constant function but only require two natural properties on F:

(1) For any two points $(x, y), (x', y') \in F$, we have $x > x' \Leftrightarrow y < y'$, namely, when the reserve of $\mathcal{X}$ increases, the reserve of $\mathcal{Y}$ decreases and vice versa;

(2) $F(x, y)$ is differentiable and the marginal exchange rate $\left|\frac{\partial F/\partial x}{\partial F/\partial y}\right|$ is decreasing with respect to x.

Most CFMMs satisfy these two properties, including Uniswap v2 and v3. The two properties imply that for any x, there is exactly one y such that $(x, y) \in F$ and vice versa. To simplify notations, we use $F_y(x)$ to denote that y such that $(x, y) \in F$ and similarly define $F_x(y)$.

We call the pool's state after the latest block its *initial state* $s_0 = (x_0, y_0)$.

Liquidity Providers. Liquidity providers contribute pairs of tokens (e.g., ETH and USDC) to a CFMM pool, which is then used to fulfill users' trade orders. In exchange, LPs earn fees from each trade in the pool. A small fraction $f \in [0, 1)$ of each trade's input tokens are charged as swap fees and are shared by liquidity providers proportional to their contribution to liquidity reserves. The fees are temporarily kept by the pool and can be withdrawn by liquidity providers by burning their liquidity tokens. In our design, the swap fees are stored separately like in Uniswap v3, rather than being continuously deposited in the pool as liquidity (e.g., Uniswap v2). Throughout this paper, we only consider swap-like transactions and assume no liquidity deposit or redemption.

Users/Noise Traders. Users are a population of noise traders who intend to trade through the CFMM pool. Each user holds one type of asset and seeks to exchange it for another by creating a swap transaction. Specifically, a transaction $\mathsf{TX} = (\mathcal{X} \to \mathcal{Y}, \delta_{\mathcal{X}}^{in}, \delta_{\mathcal{Y}}^{out})$ specifies that the user is willing to spend up to $\delta_{\mathcal{X}}^{in}$ units of asset $\mathcal{X}$, provided they receive at least $\delta_{\mathcal{Y}}^{out}$ units of asset $\mathcal{Y}$. Similarly, the signer of transaction $\mathsf{TX} = (\mathcal{Y} \to \mathcal{X}, \delta_{\mathcal{Y}}^{in}, \delta_{\mathcal{X}}^{out})$ allows at most $\delta_{\mathcal{Y}}^{in}$ units of $\mathcal{Y}$ to be taken from their account if they receive at least $\delta_{\mathcal{X}}^{out}$ units of $\mathcal{X}$. For simplicity of notations, the terms $\delta_{\mathcal{X}}^{in}$ and $\delta_{\mathcal{Y}}^{in}$ refer to the actual quantities of input tokens available for trading, after the fees have been deducted. In this paper, we use trade/order/transaction interchangeably.

Arbitrageurs/Informed Traders. Arbitrageurs are informed traders with access to external markets. They continuously monitor the price disparities between the CFMM and

external markets (e.g., CEX[2] like Binance), seeking to profit by arbitrage.

Each arbitrageur i holds a private belief v_i^* regarding the external price of the risky asset $\mathcal{X}$. These beliefs guide arbitrageurs to exploit MEV opportunities. Arbitrageurs may have different beliefs for various reasons. First, they may focus on different external markets or operate on different timelines for executing off-chain trades. Second, prices can exhibit significant volatility, leading to varying price expectations among arbitrageurs. Lastly, arbitrageurs' trading costs may be different, especially in off-chain trading venues. The individual price-belief model was also adopted by several previous work [47, 48].

3.2 RediSwap's Workflow

In today's MEV supply chains, block builders and proposers capture most MEV when arbitrageurs compete for MEV opportunities through MEV auctions [2]. From a redistribution perspective, this is undesirable. RediSwap avoids it by shifting arbitrageur competition to the CFMM side.

RediSwap consists of both an off-chain service, where the MEV-redistribution mechanism operates, and the on-chain settlement, which includes CFMM execution. The overall workflow is as follows: Users who wish to trade in the CFMM *privately* submit transactions to the off-chain service, e.g., via an RPC channel. These transactions may create MEV opportunities. Instead of exposing transaction details, RediSwap requires arbitrageurs to submit their price beliefs on the risky asset to the off-chain service. With the pending transactions, arbitrageur reports, and CFMM's current state, the off-chain service runs the MEV-redistribution mechanism to determine transaction sequencing. The output is an *MEV bundle* containing user transactions, MEV transactions, and direct transfers (from arbitrageurs to users/LPs), which is then settled on-chain when included in a block.

Crucially, the off-chain service inserts MEV transactions on behalf of arbitrageurs based on their reported beliefs. This requires arbitrageurs to grant RediSwap permission to manage token transfers on their behalf (e.g., via pre-authorization designs like Permit2 [49]), and we assume arbitrageurs authorize sufficient assets for trades (settings with insufficient assets are discussed in section 6). Regarding the on-chain settlement, the CFMM in RediSwap executes only the bundle produced by the MEV-redistribution mechanism. This design gives CFMMs control over transaction sequencing. Moreover, MEV transactions inserted by the mechanism are exempt from swap fees (namely, $f = 0$), enabling arbitrageurs to correct even tiny price discrepancies [12].

The next sections mainly focus on the MEV-redistribution mechanism, the core of this paper, while practical implementation considerations of RediSwap are discussed in section 6.

3.3 MEV-Redistribution Mechanism

The key novelty of RediSwap is a new MEV-redistribution mechanism. It determines which user transactions are included in the bundle, how to insert MEV transactions for arbitrageurs,

[2]As in previous work [6], we assume the CEX has infinite liquidity, so the trade size does not affect the price on the CEX.

in what order they are executed, and how much MEV is refunded to users and LPs. These decisions are formalized by three functions: a bundle generation rule, a payment rule, and a refund rule.

Before presenting the formal definitions, we first introduce necessary notations. We consider n arbitrageurs and use $\mathbf{q}$ to denote a n-sized vector where q_i is arbitrageur i's report on the external price of the risky asset $\mathcal{X}$. Their true belief is v_i^*, which may differ from q_i. We assume each arbitrageur i can create any number of Sybil transactions, the set of which is denoted by S_i. Thus, we use $M = R \cup S$ to denote the set of pending transactions in the CFMM pool, where R is a set of *real* transactions from users, and $S = S_1 \cup S_2 \cup \cdots \cup S_n$.

Definition 1 (Bundle Generation Rule)**.** *A bundle generation rule is a function* $\mathbf{b}$ *from the pool's initial state* s_0*, pending transactions* M*, and arbitrageurs' report* $\mathbf{q}$ *to an ordered set* B *of transactions (a bundle). The elements in* B *are* $T \cup A$*, where* $T \subseteq M$ *is a subset of pending transactions, and* $A = A_1 \cup \cdots \cup A_n$ *is the set of MEV transactions inserted by the rule on behalf of arbitrageurs.*

Definition 2 (Payment Rule)**.** *A payment rule is a function* $\mathbf{p}$ *from the pool's initial state* s_0*, pending transactions* M*, and arbitrageurs' report* $\mathbf{q}$ *to a non-negative number* $p_i(s_0, M, \mathbf{q})$ *for each arbitrageur* $i \in [n]$*.*

Definition 3 (Refund Rule)**.** *A refund rule is a function* $\mathbf{r}$ *from the pool's initial state* s_0*, pending transactions* M*, and arbitrageurs' report* $\mathbf{q}$ *to a non-negative number* $r(s_0, M, \mathbf{q}, \mathsf{TX}_j)$ *for each transaction* $\mathsf{TX}_j \in M$*; the remaining payments from arbitrageurs, i.e.,* $\sum_{i\in[n]} p_i(s_0, M, \mathbf{q}) - \sum_{\mathsf{TX}_j \in M} r(s_0, M, \mathbf{q}, \mathsf{TX}_j)$ *are refunded to liquidity providers.*

When the context is clear, we may shorten $r(s_0, M, \mathbf{q}, \mathsf{TX}_j)$ to $r(\mathsf{TX}_j)$ or r_j.

Definition 4 (MEV-Redistribution Mechanism)**.** *An MEV-redistribution mechanism is a triple* $(\mathbf{b}, \mathbf{p}, \mathbf{r})$ *in which* $\mathbf{b}$ *is a bundle generation rule,* $\mathbf{p}$ *is a payment rule, and* $\mathbf{r}$ *is a refund rule.*

3.4 Desired Properties

This section formalizes what it means for an MEV-redistribution mechanism to be game-theoretically sound. First of all, an arbitrageur should be incentivized to report their true belief in the external price of the risky asset $\mathcal{X}$ (defined as "dominant-strategy incentive compatible" below), and truthful arbitrageurs always obtain nonnegative utility (defined as "individually rational" below). Meanwhile, we must prevent arbitrageurs from stealing user refunds by posing as users in Sybil attacks. As a reminder, we assume that arbitrageurs can submit any number of Sybil transactions, which may receive some refunds. We require that inserting Sybil transactions will not harm users' utilities (defined as "Sybil-proof" below).

Definition 5 (Arbitrageur Utility Function)**.** *For an MEV-redistribution mechanism* $(\mathbf{b}, \mathbf{p}, \mathbf{r})$*, pool's initial state* s_0*, pending transactions* M*, arbitrageurs' report* $\mathbf{q}$*, and generated bundle*

B, the utility of arbitrageur i with true belief v_i^* and Sybil transactions $S_i \subseteq M$ is

$$u\big(S_i, q_i; S_{-i}, \mathbf{q}_{-i}\big) \coloneqq \sum_{\mathsf{TX}_j \in S_i \cap B} \frac{x_{j-1} - x_j}{1 - f \cdot \mathbb{1}_{\{x_j > x_{j-1}\}}} \cdot v_i^* + \frac{y_{j-1} - y_j}{1 - f \cdot \mathbb{1}_{\{y_j > y_{j-1}\}}} + r\left(\mathsf{TX}_j\right) \tag{1a}$$

$$+ \sum_{\mathsf{TX}_j \in A_i} (x_{j-1} - x_j) \cdot v_i^* + (y_{j-1} - y_j) \tag{1b}$$

$$- \; p_i, \tag{1c}$$

where $S_i \cap B$ is i's Sybil transactions included in the bundle; $A_i \subseteq B$ is the set of MEV transactions inserted on behalf of i by the bundle generation rule $\mathbf{b}$; $f \in [0, 1)$ is the fraction of swap fees charged by the pool; (x_{j-1}, y_{j-1}) and (x_j, y_j) are the pool states before and after executing TX_j, respectively; $\mathbb{1}_{\{x_j > x_{j-1}\}}$ indicates that TX_j is an $\mathcal{X} \to \mathcal{Y}$ transaction, and $\mathbb{1}_{\{y_j > y_{j-1}\}}$ indicates a $\mathcal{Y} \to \mathcal{X}$ transaction.

On the LHS of (1), we highlight the dependence of the utility function on the two arguments that are under arbitrageur i's direct control: the choices of Sybil transactions S_i and the report on external price q_i. We also explicitly show its dependence on other arbitrageurs' strategy S_{-i} and $\mathbf{q}_{-i}$. Here, $\mathbf{q}_{-i}$ means the vector $\mathbf{q}$ of all reports but with the i-th component removed, and similarly for S_{-i}. The formula (1a) sums over arbitrageur i's Sybil transactions included in the bundle. The first two terms represent i's profit from transaction execution (noting that, like other pending transactions, these Sybil transactions are subject to swap fees), while the third term is the refund received by the transaction. The formula (1b) aggregates the revenue from i's MEV transactions that the mechanism adds to the bundle. The formula (1c) is the cost i needs to pay for the MEV opportunity, as determined by the payment rule.

Definition 6 (Dominant-Strategy Incentive Compatible)**.** *An MEV-redistribution mechanism $(\mathbf{b}, \mathbf{p}, \mathbf{r})$ is dominant-strategy incentive compatible if, for every pool's initial state s_0, pending transactions $M = R \cup S_{-i}$, and other arbitrageurs' report $\mathbf{q}_{-i}$, truthfully reporting the belief (i.e., setting $q_i = v_i^*$) is always a dominant strategy for every arbitrageur i, namely,*

$$u(S_i = \emptyset, v_i^*; S_{-i}, \mathbf{q}_{-i}) \geq u(S_i = \emptyset, q_i; S_{-i}, \mathbf{q}_{-i}) \text{ for all } q_i \in \mathbb{R}.$$

Definition 7 (Individually Rational)**.** *An MEV-redistribution mechanism $(\mathbf{b}, \mathbf{p}, \mathbf{r})$ is individually rational if, for every pool's initial state s_0, pending transactions $M = R \cup S_{-i}$, and other arbitrageurs' report $\mathbf{q}_{-i}$, truthfully reporting always obtains nonnegative utility for every arbitrageur i, namely,*

$$u(S_i = \emptyset, v_i^*; S_{-i}, \mathbf{q}_{-i}) \geq 0 \text{ for all } i \in [n].$$

Note that we separate the discussion of incentive compatibility and individual rationality from Sybil-proofness, and the above definitions assume that arbitrageur i does not add Sybil transactions.

As mentioned above, arbitrageurs may steal refunds by pretending to be users and submitting fake (Sybil) transactions. An MEV-redistribution mechanism is *Sybil-proof* in that creating Sybil transactions does not reduce any user's utility.

Definition 8 (User Utility Function)**.** *For an MEV-redistribution mechanism* $(\mathbf{b}, \mathbf{p}, \mathbf{r})$*, pool's initial state* s_0*, pending transactions* $M = R \cup S$*, arbitrageurs' report* $\mathbf{q}$*, the utility of the originator of a transaction* $\mathsf{TX}_j \in R$ *is*

$$U(\mathsf{TX}_j \mid S, \mathbf{q}) = \frac{\delta_{\mathcal{Y}}}{\delta_{\mathcal{X}}} \cdot \mathcal{X}_j + \mathcal{Y}_j, \tag{2}$$

where $(\mathcal{X}_j, \mathcal{Y}_j)$ *is the number of tokens the originator has after executed through the mechanism, and* $\delta_{\mathcal{X}} = \delta^{in}_{\mathcal{X}}$, $\delta_{\mathcal{Y}} = \delta^{out}_{\mathcal{Y}}$ *if* $\mathsf{TX} = (\mathcal{X} \to \mathcal{Y}, \delta^{in}_{\mathcal{X}}, \delta^{out}_{\mathcal{Y}})$*;* $\delta_{\mathcal{X}} = \delta^{out}_{\mathcal{X}}$, $\delta_{\mathcal{Y}} = \delta^{in}_{\mathcal{Y}}$ *if* $\mathsf{TX} = (\mathcal{Y} \to \mathcal{X}, \delta^{in}_{\mathcal{Y}}, \delta^{out}_{\mathcal{X}})$.

It is assumed that before execution, the user of an $\mathcal{X} \to \mathcal{Y}$ transaction holds $\delta^{in}_{\mathcal{X}}$ units of $\mathcal{X}$ token and no $\mathcal{Y}$ token, while the user of an $\mathcal{Y} \to \mathcal{X}$ transaction has $\delta^{in}_{\mathcal{Y}}$ units of $\mathcal{Y}$ token and no $\mathcal{X}$ token. After execution, the user's utility depends on the results of the mechanism, where $\mathcal{X}_j$ and $\mathcal{Y}_j$ in eq. (2) counts not only the direct execution of TX_j but also the extra refund they receive. Additionally, we highlight the dependence of the utility function on the mechanism's inputs S and $\mathbf{q}$, which are under arbitrageurs' control, though.

Definition 9 (Sybil-proof)**.** *An MEV-redistribution mechanism* $(\mathbf{b}, \mathbf{p}, \mathbf{r})$ *is Sybil-proof if, for any initial state* s_0*, pending transactions* $M = R \cup S_{-i}$*, and arbitrageurs' report* $\mathbf{q}$*, creating Sybil transactions* S_i *will not reduce any user's utility:*

$$U(\mathsf{TX}_j \mid \emptyset \cup S_{-i}, \mathbf{q}) \geq U(\mathsf{TX}_j \mid S_i \cup S_{-i}, \mathbf{q}) \text{ for all } \mathsf{TX}_j \in R \text{ and all } S_i.$$

4 Our Mechanism

This section introduces the MEV-redistribution mechanism in RediSwap. In the mechanism design problem, we have n arbitrageurs; everyone has their own belief v^*_i of the external price. The inputs of the mechanism contain (1) pool's initial state $s_0 = (x_0, y_0)$; (2) pending transactions M; and (3) arbitrageurs' report $\mathbf{q} = (q_1, \cdots, q_n)$.

Note that arbitrageurs may misreport beliefs and/or submit Sybil transactions. So, the report q_i may differ from the true belief v^*_i for any arbitrageur $i \in [n]$. For clarity, we use $\hat{s}_i = (\hat{x}_i, \hat{y}_i) \in F$ to denote the pool state at which the spot price equals i's report q_i, i.e., the state such that $\left|\frac{\partial F/\partial x}{\partial F/\partial y}(\hat{x}_i, \hat{y}_i)\right| = q_i$. Similarly, we use $s^*_i = (x^*_i, y^*_i) \in F$ to denote the pool state at which the spot price equals i's true belief v^*_i, i.e., the state such that $\left|\frac{\partial F/\partial x}{\partial F/\partial y}(x^*_i, y^*_i)\right| = v^*_i$.

4.1 Some Notations

Before presenting our mechanism, we introduce several notations.

For any arbitrageur i who submits a report q_i, we define the *pool value* of state $s = (x, y) \in F$ by

$$\phi(s, q_i) \coloneqq (x \cdot q_i + y) - (\hat{x}_i \cdot q_i + \hat{y}_i), \tag{3}$$

where $(\hat{x}_i, \hat{y}_i)$ is the pool state at which the spot price equals q_i. Intuitively, $\phi(s, q_i)$ characterizes the profit that arbitrageur i can earn by inserting a transaction that moves the pool from state s to $(\hat{x}_i, \hat{y}_i)$. Note that when $q_i = v_i^*$, $(\hat{x}_i, \hat{y}_i)$ is the no-arbitrage state for i. In this scenario, $\phi(s_0, q_i)$ coincides with the loss-versus-rebalancing (LVR) from i's trade. In other words, it quantifies the costs incurred by liquidity providers due to the rebalancing arbitrage that corrects the pool's stale price.

When the context is clear, we write $\phi(s, q_i)$ as $\phi_i(s)$ or simply ϕ_i.

Claim 1. *For any state $s = (x, y) \in F$ and any report q_i, $\phi(s, q_i) \geq 0$.*

Proof. Fix an arbitrary report q_i, there is a unique pool state $(\hat{x}_i, \hat{y}_i) \in F$ such that $\left|\frac{\partial F/\partial x}{\partial F/\partial y}(\hat{x}_i, \hat{y}_i)\right| = q_i$. For any state $s = (x, y) \in F$, there are three cases:

Case 1: $x = \hat{x}_i$ (and thus $y = \hat{y}_i$). $\phi(s, q_i) = 0$ by definition.

Case 2: $x > \hat{x}_i$ (and thus $y < \hat{y}_i$). Recall that we work on constant functions such that the marginal exchange rate $\left|\frac{\partial F/\partial x}{\partial F/\partial y}\right|$ decreases with respect to x (see section 3.1). It implies that $q_i > \frac{\hat{y}_i - y}{x - \hat{x}_i}$. Thus, we have $\phi(s, q_i) = (x - \hat{x}_i)\left(q_i - \frac{\hat{y}_i - y}{x - \hat{x}_i}\right) > 0$.

Case 3: $x < \hat{x}_i$ and $y > \hat{y}_i$. $q_i < \frac{y - \hat{y}_i}{\hat{x}_i - x} \Rightarrow \phi(s, q_i) = (x - \hat{x}_i)\left(q_i - \frac{y - \hat{y}_i}{\hat{x}_i - x}\right) > 0$.

This finishes the proof. □

Next, for any transaction TX_j, we define its *potential value* to arbitrageur i with report q_i by

$$\Phi(\mathsf{TX}_j, q_i) \coloneqq \begin{cases} \delta_{\mathcal{X}}^{in} \cdot q_i - \delta_{\mathcal{Y}}^{out}, & \text{when } \mathsf{TX}_j = (\mathcal{X} \to \mathcal{Y}, \delta_{\mathcal{X}}^{in}, \delta_{\mathcal{Y}}^{out}); \\ -\delta_{\mathcal{X}}^{out} \cdot q_i + \delta_{\mathcal{Y}}^{in}, & \text{when } \mathsf{TX}_j = (\mathcal{Y} \to \mathcal{X}, \delta_{\mathcal{Y}}^{in}, \delta_{\mathcal{X}}^{out}). \end{cases} \tag{4}$$

Note that $\Phi(\mathsf{TX}_j, q_i) < 0$ if and only if in two cases: (1) $\mathsf{TX}_j = (\mathcal{X} \to \mathcal{Y}, \delta_{\mathcal{X}}^{in}, \delta_{\mathcal{Y}}^{out})$ and $q_i < \delta_{\mathcal{Y}}^{out}/\delta_{\mathcal{X}}^{in}$; (2) $\mathsf{TX}_j = (\mathcal{Y} \to \mathcal{X}, \delta_{\mathcal{Y}}^{in}, \delta_{\mathcal{X}}^{out})$ and $q_i > \delta_{\mathcal{Y}}^{in}/\delta_{\mathcal{X}}^{out}$.

Based on $\Phi(\mathsf{TX}_j, q_i)$, we further define the *actual value* of transaction TX_j to arbitrageur i as

$$V(\mathsf{TX}_j, q_i) = \max\left\{0, \Phi(\mathsf{TX}_j, q_i)\right\}. \tag{5}$$

Sometimes, we abbreviate $\Phi(\mathsf{TX}_j, q_i)$ and $V(\mathsf{TX}_j, q_i)$ as $\Phi_i(\mathsf{TX}_j)$ and $V_i(\mathsf{TX}_j)$, respectively.

The definition of $\Phi(\mathsf{TX}_j, q_i)$ reflects the worst-case execution of TX_j for the user, which is the best scenario for arbitrageurs because, intuitively, the user of TX_j and arbitrageurs form a zero-sum game. This worst-case outcome occurs when TX_j is executed at its *limit state*, denoted by $s_j^\ell = (x_j^\ell, y_j^\ell)$. At this state, TX_j pays exactly the maximum amount of input tokens and receives the minimum amount of output tokens. That is, executing TX_j at (x_j^ℓ, y_j^ℓ) transitions the pool to $(x_j^\ell + \Delta x_j, y_j^\ell + \Delta y_j)$, incurring a net impact of $(\Delta x_j, \Delta y_j)$ given by

$$(\Delta x_j, \Delta y_j) \coloneqq \begin{cases} (\delta_{\mathcal{X}}^{in}, -\delta_{\mathcal{Y}}^{out}), & \text{when } \mathsf{TX}_j = (\mathcal{X} \to \mathcal{Y}, \delta_{\mathcal{X}}^{in}, \delta_{\mathcal{Y}}^{out}); \\ (-\delta_{\mathcal{X}}^{out}, \delta_{\mathcal{Y}}^{in}), & \text{when } \mathsf{TX}_j = (\mathcal{Y} \to \mathcal{X}, \delta_{\mathcal{Y}}^{in}, \delta_{\mathcal{X}}^{out}). \end{cases} \tag{6}$$

It is straightforward to verify that $\Phi_i(\mathsf{TX}_j) = \Delta x_j \cdot q_i + \Delta y_j$.

4.2 Mechanism Description

We are now ready to describe our mechanism. Intuitively, each pending transaction $\mathsf{TX}_j \in M$ *and* the initial state s_0 may generate MEV. The proposed mechanism auctions off these MEV opportunities separately. At a high level, the mechanism uses the arbitrageurs' reports to generate "bids" for each arbitrageur i: $\Phi_i(\mathsf{TX}_j)$ for each transaction TX_j and $\phi_i(s_0)$ for the initial state. It then conducts $|M|+1$ simultaneous sealed-bid second-price auctions, one for each potential MEV opportunity.

- **Bundle generation rule:** Our mechanism constructs the bundle following Algorithm 1, which consists of two parts. In the first part (lines 1 - 21) each pending transaction $\mathsf{TX}_j \in M$ is processed in turn. Each iteration starts with computing TX_j's limit state (x_j^ℓ, y_j^ℓ) and the corresponding impact on the pool $(\Delta x_j, \Delta y_j)$ by eq. (6). Then, it computes $\Phi_i(\mathsf{TX}_j)$ for each arbitrageur $i \in [n]$ as her bid and selects the winner w_j of this iteration who values TX_j the most, i.e., $w_j = \arg\max_{i\in[n]} \Phi_i(\mathsf{TX}_j)$. If the winning bid $\Phi_{w_j}(\mathsf{TX}_j) < 0$, the mechanism skips this transaction. Otherwise, it assigns TX_j to the winner w_j by constructing a "sandwich" on her behalf. Specifically, the mechanism inserts a front-running transaction for w_j from (x_0, y_0) to (x_j^ℓ, y_j^ℓ) so that TX_j executes at its limit state, followed by a back-running transaction from the post-execution state $(x_j^\ell + \Delta x_j, y_j^\ell + \Delta y_j)$ to the initial state (x_0, y_0). Overall, the "sandwich" is:

$$(x_0, y_0) \xrightarrow{\mathsf{TX}_{w_j}^{\mathsf{Front}}} (x_j^\ell, y_j^\ell) \xrightarrow{\mathsf{TX}_j} (x_j^\ell + \Delta x_j, y_j^\ell + \Delta y_j) \xrightarrow{\mathsf{TX}_{w_j}^{\mathsf{Back}}} (x_0, y_0).$$

  After enumerating all pending transactions, the second part of Algorithm 1 (lines 22 - 28) sells the rebalancing arbitrage opportunity from the initial state s_0. For each arbitrageur $i \in [n]$, it computes $\phi_i(s_0)$ as her bid and selects the winner $w' = \arg\max_{i\in[n]} \phi_i(s_0)$. Specifically, the mechanism adds an arbitrage transaction for w' to move the pool from (x_0, y_0) to $(\hat{x}_{w'}, \hat{y}_{w'})$, which is the no-arbitrage state for w' according to her report.

- **Payment rule:** By construction, there are $|M|+1$ winners (note that the same arbitrageur can win multiple times), corresponding to $|M|$ transactions and the initial state. The payment rule requires each winner to pay the second highest value in units of the numéraire $\mathcal{Y}$. Concretely: For each transaction TX_j, the winner w_j pays $\max_{i\neq w_j} V_i(\mathsf{TX}_j)$, and everyone else pays zero; For the rebalancing arbitrage, the winner w' pays $\max_{i\neq w'} \phi_i(s_0)$, and the others pay zero. By the definition of $V_i(\mathsf{TX}_j)$ and Claim 1, each payment is non-negative. Overall, an arbitrageur's total payment is the sum of her individual payments across these $|M|+1$ MEV opportunities.

- **Refund rule:** All payments are refunded to users and liquidity providers. Specifically, the owner of transaction TX_j receives $\max_{i\neq w_j} V_i(\mathsf{TX}_j)$, and the LPs receive $\max_{i\neq w'} \phi_i(s_0)$.

Note that these payments and refunds are realized by inserting direct transfer transactions from the winning arbitrageurs to users and liquidity providers. Specifically, refunds to LPs are sent to the liquidity pools as fees, which requires CFMMs to support such direct

Algorithm 1: Bundle Generation Rule of the Mechanism

Input: An initial state $s_0 = (x_0, y_0)$, a set of pending transactions M, and arbitrageurs' reports $\mathbf{q} = (q_1, \cdots, q_n)$, each corresponding to a state $\hat{s}_i = (\hat{x}_i, \hat{y}_i)$.

Output: A bundle to be executed by the CFMM.

for *each* $j \in [1 : |M|]$ **do**
  if $\mathsf{TX}_j = (\mathcal{X} \to \mathcal{Y}, \delta^{in}_{\mathcal{X}}, \delta^{out}_{\mathcal{Y}})$ **then**
    (x^ℓ_j, y^ℓ_j) is the limit state satisfying $y^\ell_j - F_y(x^\ell_j + \delta^{in}_{\mathcal{X}}) = \delta^{out}_{\mathcal{Y}}$.
    Let $\Delta x \leftarrow \delta^{in}_{\mathcal{X}}$, $\Delta y \leftarrow -\delta^{out}_{\mathcal{Y}}$.
  else if $\mathsf{TX}_j = (\mathcal{Y} \to \mathcal{X}, \delta^{in}_{\mathcal{Y}}, \delta^{out}_{\mathcal{X}})$ **then**
    (x^ℓ_j, y^ℓ_j) is the limit state satisfying $x^\ell_j - F_x(y^\ell_j + \delta^{in}_{\mathcal{Y}}) = \delta^{out}_{\mathcal{X}}$.
    Let $\Delta x \leftarrow -\delta^{out}_{\mathcal{X}}$, $\Delta y \leftarrow \delta^{in}_{\mathcal{Y}}$.
  for *each* $i \in [n]$ **do**
    Let $\Phi_i(\mathsf{TX}_j) \leftarrow \Delta x \cdot q_i + \Delta y$.
  Let $\Phi_{w_j} \leftarrow \max_{i \in [n]} \Phi_i(\mathsf{TX}_j)$. `// Arbitrageur` w_j `values` TX_j `the most`
  if $\Phi_{w_j} \geq 0$ **then**
    if $x_0 < x^\ell_j$ **then**
      Insert a transaction on behalf of arbitrageur w_j
      $\mathsf{TX} = \big(\mathcal{X} \to \mathcal{Y}, \delta^{in}_{\mathcal{X}} = x^\ell_j - x_0, \delta^{out}_{\mathcal{Y}} = y_0 - y^\ell_j\big)$.
    else if $x_0 > x^\ell_j$ **then**
      Insert a transaction on behalf of arbitrageur w_j
      $\mathsf{TX} = \big(\mathcal{Y} \to \mathcal{X}, \delta^{in}_{\mathcal{Y}} = y^\ell_j - y_0, \delta^{out}_{\mathcal{X}} = x_0 - x^\ell_j\big)$.
    Place the user transaction TX_j.
    Let the current state $(x', y') \leftarrow (x^\ell_j + \Delta x, y^\ell_j + \Delta y)$.
    if $x' < x_0$ **then**
      Insert a transaction on behalf of arbitrageur w_j
      $\mathsf{TX} = \big(\mathcal{X} \to \mathcal{Y}, \delta^{in}_{\mathcal{X}} = x_0 - x', \delta^{out}_{\mathcal{Y}} = y' - y_0\big)$.
    else if $x' > x_0$ **then**
      Insert a transaction on behalf of arbitrageur w_j
      $\mathsf{TX} = \big(\mathcal{Y} \to \mathcal{X}, \delta^{in}_{\mathcal{Y}} = y_0 - y', \delta^{out}_{\mathcal{X}} = x' - x_0\big)$.
for *each* $i \in [n]$ **do**
  Let $\phi_i \leftarrow (x_0 \cdot q_i + y_0) - (\hat{x}_i \cdot q_i + \hat{y}_i)$.
Let $\phi_{w'} \leftarrow \max_{i \in [n]} \phi_i$. `// Arbitrageur` w' `values` LVR `the most`
if $x_0 < \hat{x}_{w'}$ **then**
  Add a transaction on behalf of arbitrageur w'
  $\mathsf{TX} = \big(\mathcal{X} \to \mathcal{Y}, \delta^{in}_{\mathcal{X}} = \hat{x}_{w'} - x_0, \delta^{out}_{\mathcal{Y}} = y_0 - \hat{y}_{w'}\big)$.
else if $x_0 > \hat{x}_{w'}$ **then**
  Add a transaction on behalf of arbitrageur w'
  $\mathsf{TX} = \big(\mathcal{Y} \to \mathcal{X}, \delta^{in}_{\mathcal{Y}} = \hat{y}_{w'} - y_0, \delta^{out}_{\mathcal{X}} = x_0 - \hat{x}_{w'}\big)$.

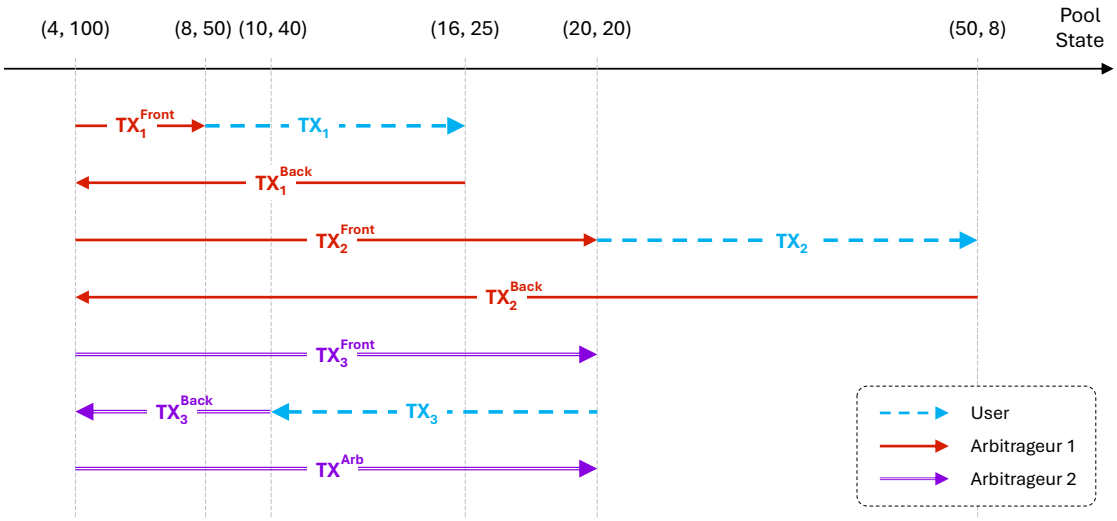


Figure 1: An illustration of the bundle constructed by our mechanism for Example 1. Pool states (x, y) in the figure are shown in ascending order based on the value of x. Dotted lines represent pending transactions from users, while solid lines represent MEV transactions from arbitrageurs.

donations. All transfer transactions are appended to the bundle constructed by the bundle generation rule.

Here, we use Example 1 to showcase how the above mechanism operates.

Example 1. *Consider a CFMM with the trading curve* $F(x, y) = xy = 400$*, an initial state* $s_0 = (4, 100)$*, and a swap fee* $f = 0$*. There are three pending transactions:* $\mathsf{TX}_1 = (\mathcal{X} \to \mathcal{Y}, \delta_{\mathcal{X}}^{in} = 8, \delta_{\mathcal{Y}}^{out} = 25)$*,* $\mathsf{TX}_2 = (\mathcal{X} \to \mathcal{Y}, \delta_{\mathcal{X}}^{in} = 30, \delta_{\mathcal{Y}}^{out} = 12)$*, and* $\mathsf{TX}_3 = (\mathcal{Y} \to \mathcal{X}, \delta_{\mathcal{Y}}^{in} = 20, \delta_{\mathcal{X}}^{out} = 10)$*. Two arbitrageurs hold a different belief about the external price of* $\mathcal{X}$*, with values of* $v_1^* = 4$ *and* $v_2^* = 1$*, corresponding to the pool states* $(10, 40)$ *and* $(20, 20)$*, respectively. Both players report their beliefs truthfully, leading to the following outcomes:*

Arbitrageur i	q_i	$\phi_i(s_0)$	$V_i(\mathsf{TX}_1)$	$V_i(\mathsf{TX}_2)$	$V_i(\mathsf{TX}_3)$
1	4	36	**7**	**108**	0
2	1	**64**	0	18	**10**

According to our mechanism, arbitrageur 1 wins the MEV opportunity from TX_1 *and* TX_2*, while arbitrageur 2 wins the MEV opportunity from* TX_3 *and the initial state* s_0*. To be specific, the bundle generation rule forms a bundle as shown in Figure 1. Additionally, arbitrageur 1 pays* 18*, which is finally refunded to the owner of* TX_2*; arbitrageur 2 pays* 36*, which is refunded to LPs.*

4.3 Some Highlights of the Mechanism

Structure of the mechanism. The mechanism can be viewed as $|M| + 1$ simultaneous auctions, each selling a distinct MEV opportunity. These auctions are both separate and

linked. On one hand, each auction's outcome (i.e., the constructed bundle, payment, and refund) is determined solely by the objective details of the corresponding item (a transaction or the initial state) and the given report $\mathbf{q}$. Consequently, the outcome of one auction does not influence or rely on the others. On the other hand, all auctions rely on the same set of arbitrageur reports. This shared input links the auctions, meaning they are not fully independent; rather, they are interconnected through the common report vector that shapes their outcomes.

Construction of the bundle. The mechanism realizes the MEV opportunities by inserting MEV transactions to generate revenue for winners. However, due to the "ripple effect"[3] in CFMMs — where each transaction changes the pool state and thereby affects all subsequent transactions — enabling *all* winners to simultaneously capture their MEV value, i.e., $V_{w_j}(\mathsf{TX}_j)$ or $\phi_{w'}(s_0)$, is non-trivial. Our bundle generation rule addresses this challenge by constructing a special "sandwich" for each transaction, one that always starts from the initial state and eventually returns to the initial state again. Consequently, the final bundle comprises $|M|+1$ independent sub-bundles: a "sandwich" for each transaction and a single rebalancing arbitrage sub-bundle for the initial state (if there is no winner, the sub-bundle is empty). Crucially, these sub-bundles do not interfere with one another, allowing all auctions to occur simultaneously.

Dynamic true valuation in the auction. The process of selling each MEV opportunity closely resembles a sealed-bid second-price auction, with the key distinction that "bids" are computed by the mechanism based on the reports $\mathbf{q}$ submitted by arbitrageurs. This design leads to an interesting phenomenon: misreporting not only alters an arbitrageur's bid (and potentially the winner) but also may change her true valuation if she wins. Notably, in a standard second-price auction (as well as most classic auction settings), a bidder's true valuation of the auctioned item is fixed. In contrast, in our mechanism, particularly in the auction for the initial state, an arbitrageur's report influences how the bundle generation rule inserts MEV transactions for her upon winning, thereby shaping her actual revenue (analogous to the true valuation). This interdependence makes the game strategically more complex and interesting than in a standard second-price auction.

4.4 Properties of the Mechanism

Theorem 1. *The mechanism is DSIC.*

Theorem 2. *The mechanism is individually rational.*

Theorem 3. *The mechanism is Sybil-proof.*

Due to space constraints, we postpone the proofs to Appendix B.1 and B.2.

[3] In the CFMM context, transactions are executed sequentially. The execution of a transaction in the bundle impacts not only its own outcome (and its owner's utility) but also the pool state, which then affects all subsequent transactions. This creates a "ripple effect," where a change in one transaction can cascade through the pool and impact all behind, which complicates the task of managing multiple arbitrageurs' MEV within a single bundle.

In essence, Theorem 3 states that, under any reports (which may differ from the true beliefs), Sybil transactions do not reduce any user's total utility, i.e., the utility from transaction execution plus refunds. Recall that refunds are paid in the numéraire $\mathcal{Y}$. By treating refunds as part of the transaction outcome, we define the *overall* execution price of transaction $\mathsf{TX}_j \in M$ (if included in the bundle) as follows:

$$\begin{cases} \left(\delta^{out}_{\mathcal{Y}} + r(\mathsf{TX}_j)\right)/\delta^{in}_{\mathcal{X}}, & \text{when } \mathsf{TX}_j = (\mathcal{X} \to \mathcal{Y}, \delta^{in}_{\mathcal{X}}, \delta^{out}_{\mathcal{Y}}); \\ \left(\delta^{in}_{\mathcal{Y}} - r(\mathsf{TX}_j)\right)/\delta^{out}_{\mathcal{X}}, & \text{when } \mathsf{TX}_j = (\mathcal{Y} \to \mathcal{X}, \delta^{in}_{\mathcal{Y}}, \delta^{out}_{\mathcal{X}}), \end{cases} \tag{7}$$

where $r(\mathsf{TX}_j)$ is the refund to TX_j. Then we have the following observation.

Observation 1. *Fix an arbitrary report vector* $\mathbf{q}$*. Without loss of generality, assume that* $q_1 \geq q_2 \geq \cdots \geq q_n$*. For any pending transaction* $\mathsf{TX}_j \in M$*, the winner is either arbitrageur* 1 *or arbitrageur* n*:*

- *For* $\mathsf{TX}_j = (\mathcal{X} \to \mathcal{Y}, \delta^{in}_{\mathcal{X}}, \delta^{out}_{\mathcal{Y}})$*, the winner is always arbitrageur* 1*. If the limit price* $\delta^{out}_{\mathcal{Y}}/\delta^{in}_{\mathcal{X}} > q_1$*,* TX_j *is not included in the bundle (i.e., empty sub-bundle). Otherwise, its* overall *execution price is* $\max\left\{q_2, \delta^{out}_{\mathcal{Y}}/\delta^{in}_{\mathcal{X}}\right\}$ *if* $n \geq 2$*, or* $\delta^{out}_{\mathcal{Y}}/\delta^{in}_{\mathcal{X}}$ *if* $n = 1$*.*
- *For* $\mathsf{TX}_j = (\mathcal{Y} \to \mathcal{X}, \delta^{in}_{\mathcal{Y}}, \delta^{out}_{\mathcal{X}})$*, the winner is always arbitrageur* n*. If the limit price* $\delta^{in}_{\mathcal{Y}}/\delta^{out}_{\mathcal{X}} < q_n$*,* TX_j *is not included. Otherwise, its* overall *execution price is* $\min\left\{q_{n-1}, \delta^{in}_{\mathcal{Y}}/\delta^{out}_{\mathcal{X}}\right\}$ *if* $n \geq 2$*, or* $\delta^{in}_{\mathcal{Y}}/\delta^{out}_{\mathcal{X}}$ *if* $n = 1$*.*

As shown above, the overall outcome of a transaction is jointly determined by the arbitrageurs' reports and its own limit price, which acts like a reserve price. We defer the proof to Appendix B.3.

Resilience to simultaneous misbehavior. We have shown the desired properties. However, there is a very subtle issue: Submitting Sybil transactions can increase an arbitrageur's own utility. While this does not contradict Theorem 3, it may introduce an incentive for arbitrageurs to misreport their beliefs, which in turn could affect users' and LPs' utilities. We illustrate this point in detail with the following example:

Example 2. *Consider a CFMM with the trading curve* $F(x, y) = xy = 400$*, an initial state* $s_0 = (4, 100)$*, and a swap fee* $f = 0$*. Three arbitrageurs have true beliefs* $v^*_1 = 4$*,* $v^*_2 = 1$*, and* $v^*_3 = 0.16$*, corresponding to pool states* $(10, 40)$*,* $(20, 20)$*, and* $(50, 8)$*, respectively. There is a single pending transaction* $\mathsf{TX} = (\mathcal{X} \to \mathcal{Y}, \delta^{in}_{\mathcal{X}} = 10, \delta^{out}_{\mathcal{Y}} = 1)$*.*

(1) Without Sybil Transactions. If no Sybil transactions are submitted, each arbitrageur's dominant strategy is to report truthfully (Theorem 1), i.e., setting $q_i = v^*_i$ *for* $i \in [1:3]$*. The outcome is:*

- *The winner of* TX *is arbitrageur* 1*, and its overall execution price (by Observation 1) is* $\max\left\{q_2 = 1, \delta^{out}_{\mathcal{Y}}/\delta^{in}_{\mathcal{X}} = 1/10\right\} = 1$*.*
- *The winner of the initial-state rebalancing is arbitrageur* 3*, and the refund to liquidity providers is* $\max_{i\in\{1,2\}} \phi_i(s_0) = \phi_2(s_0) = 64$*.*

Consequently, arbitrageur 2's utility is zero.

(2) *Introducing a Sybil Transaction. Now, suppose arbitrageur 2 submits a Sybil transaction* $\mathsf{TX}' = (\mathcal{Y} \to \mathcal{X}, \delta^{in}_{\mathcal{Y}} = 20, \delta^{out}_{\mathcal{X}} = 25)$. *This does not alter the above outcomes, consistent with Theorem 3, but arbitrageur 2 gains 5 utility from* TX', *as she receives 25* $\mathcal{X}$ *at a cost of 20* $\mathcal{Y}$.

(3) *Sybil + Misreporting. To further increase utility, arbitrageur 2 misreports her belief as* $q_2 = 0.25$, *which corresponds to the pool state* $(40, 10)$. *Assume arbitrageur 1 and 3 remain truthful (i.e.,* $q_1 = 4$ *and* $q_3 = 0.16$*). The outcome now changes as follows:*

- *The winner of* TX *is still arbitrageur 1 but its overall execution price drops to 0.25.*
- *Arbitrageur 3 still wins the initial-state rebalancing, but the refund to LPs rises to 81.*
- *The Sybil transaction* TX' *executes at a better price of 0.25 due to the misreporting.*

Now, arbitrageur 2 receives 25 $\mathcal{X}$ *and an additional 13.75* $\mathcal{Y}$ *while still paying 20* $\mathcal{Y}$. *Consequently, arbitrageur 2's utility further increases to 18.75.*

The above example demonstrates that arbitrageurs may submit Sybil transactions to increase utility, which can, in turn, motivate them to misreport (such as arbitrageur 2 in Example 2). Our ultimate goal is to incentivize all arbitrageurs to truthfully report their beliefs so that users get their deserved utilities under our mechanism. We accomplish this by showing that there is a strategy profile (v^*, S) forming a Nash equilibrium, under the assumption that each arbitrageur has a *Sybil budget*, denoted by $(b^{\mathcal{X}}_i, b^{\mathcal{Y}}_i)$. This budget captures the maximum amount of $\mathcal{X}$ and $\mathcal{Y}$ tokens that an arbitrageur can commit to Sybil attacks. Additionally, we assume that each arbitrageur's belief is drawn from some known distribution $\mathcal{D}_i$. We use $\mathcal{D}$ to denote the collection of all $\mathcal{D}_i$.

In particular, we show the following theorem:

Theorem 4. *There is a Sybil strategy* $S_i(v^*_i, b^{\mathcal{X}}_i, b^{\mathcal{Y}}_i, \mathcal{D})$ *such that, under our mechanism, using* (v^*_i, S_i) *is a Nash equilibrium for every arbitrageur* $i \in [n]$.

We postpone the proof to Appendix B.4. The high-level idea is as follows. In Example 2, suppose arbitrageur 2 is fully informed of the other players' beliefs. Her best response is to misreport $q_2 = v^*_3 + \epsilon$, such that her Sybil transaction TX' benefits from the largest refund and thus the best overall execution price. However, if she also has a Sybil budget to submit an $\mathcal{X} \to \mathcal{Y}$ Sybil transaction as well, she might want to set $q_2 = v^*_1 - \epsilon$ to maximize the utility from this transaction — creating a conflict in how she should misreport. Our key observation is that rather than misreporting, the arbitrageur can equivalently manipulate the outcome by strategically setting the *limit price* in each of her Sybil transactions, which serves as the reserve price in light of Observation 1. Concretely, in Example 2, assume arbitrageur 2 has a Sybil budget $(b^{\mathcal{X}}_2, b^{\mathcal{Y}}_2) = (10, 20)$. She can maximize her utility by *truthfully reporting*, and submitting two Sybil transactions: $(\mathcal{X} \to \mathcal{Y}, \delta^{in}_{\mathcal{X}} = b^{\mathcal{X}}_2 = 10, \delta^{out}_{\mathcal{Y}} = 40)$ such that its limit price $\delta^{out}_{\mathcal{Y}}/\delta^{in}_{\mathcal{X}} = v^*_1$, and $(\mathcal{Y} \to \mathcal{X}, \delta^{in}_{\mathcal{Y}} = b^{\mathcal{Y}}_2 = 20, \delta^{out}_{\mathcal{X}} = 125)$ such that its limit price $\delta^{in}_{\mathcal{Y}}/\delta^{out}_{\mathcal{X}} = v^*_3$. We generalize this idea to show that as long as the arbitrageurs can figure

out how to correctly set the limit price (such as in the Bayesian setting), then they have no incentive to misreport. Thus the arbitrageurs will truthfully report their beliefs, and we achieve our ultimate goal that users get their deserved utilities under our mechanism.

5 Evaluation

In this section, we evaluate the efficiency of RediSwap—specifically, whether it improves execution results (i.e., price) for users and reduces loss for liquidity providers. For users, we compare RediSwap to two notable application-level solutions, UniswapX and CoWSwap, to evaluate if orders filled through RediSwap achieve better execution prices than in UniswapX or CoWSwap. For LPs, we compare the LVR of Uniswap v2 LPs with and without the MEV-redistribution mechanism.

5.1 Evaluation Methodology

Assumptions. We assume that arbitrageurs have sufficient capital to execute trades between CEXs and DEXs, making it potentially profitable for them to use their own assets to fulfill users' orders. Additionally, the liquidity on CEXs is ample enough to ensure that these arbitrage activities do not materially affect the off-chain price. We justify these assumptions in two ways: First, many arbitrageurs engage in CEX-DEX arbitrage in practice [16]; second, the liquidity on CEXs is generally substantial, as indicated by their high trading volumes (e.g., the 24-hour trading volume of ETH on Binance is $8 billion as of October 5th, 2024 [50]). Besides, for simplicity, we assume that the gas usage for a transaction of an order in UniswapX or CoWSwap is the same when the order is filled by RediSwap. We also assume that the priority fee paid by arbitrageurs in RediSwap is up to 1 Gwei. We note that this is a reasonable assumption because the arbitrageurs in RediSwap compete for the opportunity to fulfill orders at the CFMM side instead of participating in auctions at the block producer side.

Distribution of arbitrageurs' beliefs. To simulate arbitrageurs' beliefs in external prices, we obtain historical token price data from Binance [51] over a one-year period, from September 1st, 2023, to August 31st, 2024. Since not all tokens are traded on CEXs, our evaluation focuses on the eight popular tokens: BTC, ETH, USDC, USDT, DAI, LINK, MATIC, and PEPE. Specifically, we obtain candlestick data from Binance with one-second intervals for these tokens. The arbitrageurs' beliefs about a token's price in a block are simulated by a distribution between the highest and lowest prices of the token during the time slot of the block (the specific type of distribution and the number of arbitrageurs are discussed in each experiment).

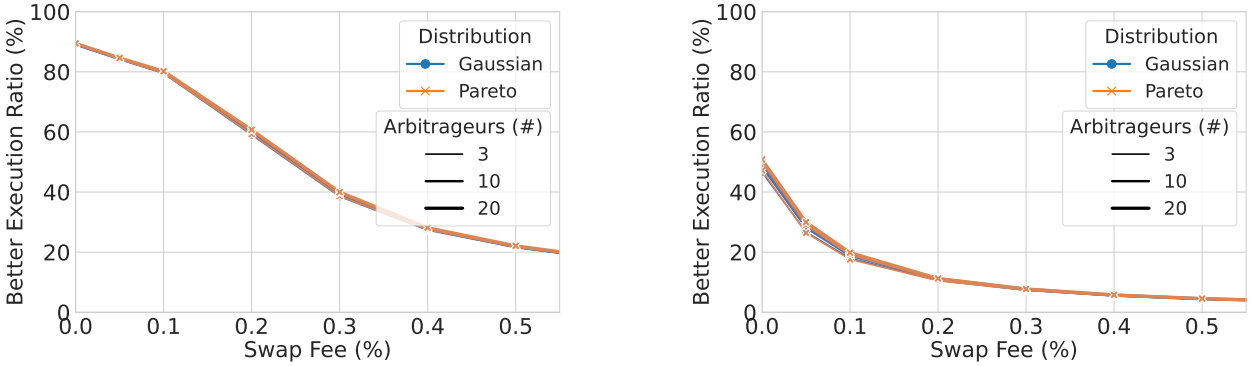
Historical trades. We collect all orders on UniswapX and CoWSwap within the same time range. Specifically, we gather orders from the DEX Analytics Platform [52] and cross-check them against Dune records [53]. Our evaluation focuses on orders where either the buy or sell side is a stablecoin (USDC, USDT, or DAI), while the counterpart is a token for which we have price data from Binance. In the end, we collect 344,936 orders in UniswapX and

100,618 orders in CoWSwap for comparison. To evaluate the efficiency in reducing LVR, we also collect the state of two Uniswap v2 pools (WETH-USDC and WETH-USDT) in each block over the same time range.

5.2 Results

Better execution prices. We replay historical trades to compute the execution price on RediSwap and compare it with the actual execution price on UniswapX or CoWSwap. In the simulation, we vary the number of participating arbitrageurs in RediSwap and explore various belief distributions that arbitrageurs might have for a token. We model searchers' beliefs using a Gaussian distribution [54] with a centered mean and controlled standard deviation, as well as a Pareto distribution [55] with a shape parameter $\alpha = 1.5$, where most arbitrageurs expect lower token valuations, but a few anticipate significantly higher ones.

Additionally, since liquidity pools can charge different swap fees, we tested swap fee settings ranging from 0 to 0.5% (we excluded 1% because the token pairs we focus on are typically concentrated in pools with lower fees [56]). Note that the information on swap fees is not available from historical trades on UniswapX and CoWSwap, as those orders may not involve a pool (e.g., some are filled using solvers' assets; see appendix A).

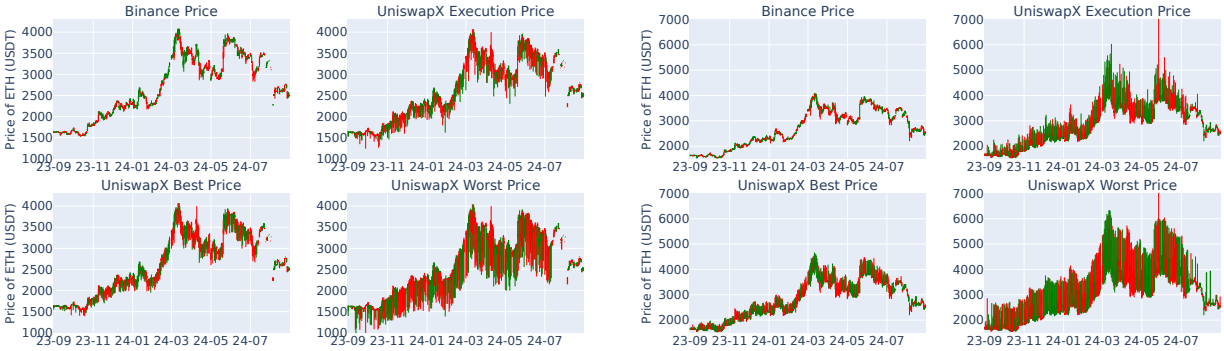


(a) UniswapX (b) CoWSwap

Figure 2: Percentage of orders with execution prices better than those on UniswapX or CoWSwap. Each marker represents the overall result under different settings of arbitrageurs and swap fees.

As shown in Figure 2, we find that execution prices in RediSwap are better than those in UniswapX for 89% of orders when the swap fee is 0. The percentage of orders with better execution prices decreases as the swap fee increases, but it remains 40% when the swap fee is 0.3%. Compared to CoWSwap, our mechanism can provide nearly equally competitive execution prices when the swap fee is 0. The distribution of prices and the number of arbitrageurs have no obvious impact on the results. This may be due to the narrow price range on Binance, which affects visible differences.

To understand why RediSwap outperforms UniswapX in most orders, we select ETH/USDT — the most frequently traded token pair in our dataset — to compare price differences on

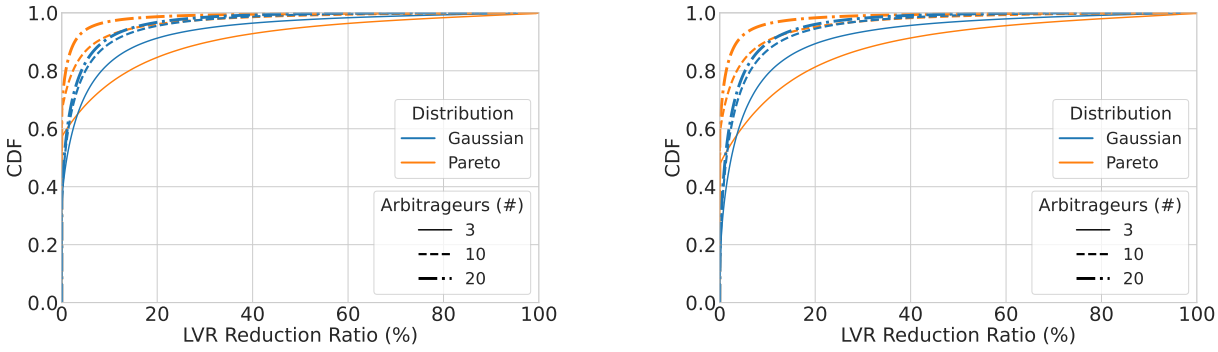


(a) Swap ETH for USDT

(b) Swap USDT for ETH

Figure 3: Candlestick charts of Binance and UniswapX ETH/USDT prices over time. Missing data indicates that our dataset does not contain relevant orders on UniswapX for those days.

Binance and UniswapX over time. For UniswapX, we analyze the execution price, the best price (the starting price of UniswapX's Dutch auction), and the worst price (the ending price in the auction, representing the lowest price users are willing to accept to fulfill orders). As shown in Figure 3, we observe that prices on Binance are better than the best price on UniswapX in most cases: for swapping ETH for USDT, the price of ETH on Binance is higher, and for swapping USDT for ETH, the price of ETH on Binance is lower. Given that arbitrageurs' beliefs follow a distribution around the prices on Binance in our simulation, RediSwap can provide users with better execution prices by leveraging the more favorable prices on CEX.



(a) WETH-USDC (Uniswap v2)

(b) WETH-USDT (Uniswap v2)

Figure 4: The CDF of LVR reduction for WETH-USDC and WETH-USDT using RediSwap across arbitrageur numbers and price distributions. A smaller LVR reduction ratio on the x-axis indicates greater efficiency.

Reducing LVR. For each block in our evaluation period, we computed the LVR incurred by two Uniswap v2 liquidity pools (WETH-USDC and WETH-USDT) when the arbitrageurs

perform CEX-DEX arbitrages, with and without using RediSwap. By dividing the loss under RediSwap by the LVR without RediSwap, we can evaluate the proportion of LVR reduction achieved by RediSwap. Similar to the previous evaluation, we also test different settings for the number of arbitrageurs and the distribution of their price beliefs for a token.

As shown in Figure 4, we can see that RediSwap effectively reduces LVR for both liquidity pools, and the reduction improves as more arbitrageurs participate. The heightened competition drives arbitrageur bids closer, ultimately minimizing LVR. For example, in over half of the blocks, the LPs in the WETH-USDC pool will only suffer at most 0.5% of the LVR they would experience without using RediSwap, when there are 20 arbitrageurs with beliefs following a Gaussian distribution.

6 Conclusion and Discussion

We have introduced RediSwap, a CFMM aided by an MEV-redistribution mechanism that can capture MEV at the application level and then refund it to the relevant participants (e.g., users and liquidity providers in this paper). We proved that the MEV-redistribution mechanism is truthful and Sybil-proof. Moreover, RediSwap is designed to not rely on arbitrageurs' sophistication. Below, we discuss implementation considerations and future directions.

"Skipped" Transactions. Recall that a pending transaction will be skipped by the mechanism and thus not be included in the bundle if it is not profitable for any arbitrageur. Such transactions are not targeted by our mechanism — as they would not have been exploited even without the mechanism — and thus do not undermine our goal of protecting users from adverse MEV extraction. For completeness, these "skipped" transactions can be appended to the end of the output bundle following a certain ordering policy, although their execution is not guaranteed.

Price Belief. The model assumes each arbitrageur i has a single valuation v_i for the risky asset $\mathcal{X}$. In practice, however, an arbitrageur may have different beliefs about the price at which she would sell versus buy the asset, e.g., due to the bid-ask spread on centralized exchanges. To capture this more general scenario, a direct trial is to modify arbitrageur i's utility function as follows:

$$
\begin{aligned}
u\big(S_i, q_i; S_{-i}, \mathbf{q}_{-i}\big) \coloneqq & \sum_{\mathsf{TX}_j \in S_i \cap B} \Bigg[\mathbb{1}_{\{x_j < x_{j-1}\}} \cdot \left((x_{j-1} - x_j) \cdot v_{i,0} + \frac{y_{j-1} - y_j}{1-f} \right) \\
& \qquad + \mathbb{1}_{\{x_j > x_{j-1}\}} \cdot \left(\frac{x_{j-1} - x_j}{1-f} \cdot v_{i,1} + (y_{j-1} - y_j) \right) + r\left(\mathsf{TX}_j\right) \Bigg] \\
+ & \sum_{\mathsf{TX}_j \in A_i} \Big[\mathbb{1}_{\{x_j < x_{j-1}\}} \cdot \left((x_{j-1} - x_j) \cdot v_{i,0} + (y_{j-1} - y_j) \right) \\
& \qquad + \mathbb{1}_{\{x_j > x_{j-1}\}} \cdot \left((x_{j-1} - x_j) \cdot v_{i,1} + (y_{j-1} - y_j) \right) \Big] \\
- & \; p_i,
\end{aligned}
$$

where $v_{i,0}$ and $v_{i,1}$ are the price at which i can sell and buy $\mathcal{X}$ on the external market, respectively. Correspondingly, arbitrageur i's report can be extended to $q_i = (q_{i,0}, q_{i,1})$,

and one can adjust the definitions of $\phi(s, q_i)$ and $\Phi(\mathsf{TX}_j, q_i)$ and further the mechanism accordingly, which we show in Appendix C. We claim that the modified mechanism remains DSIC, IR, and Sybil-proof under the above utility function.

However, this modified utility function is not entirely accurate in the two-belief setting. It computes the profit for each transaction separately and immediately upon execution — as if arbitrageur i sells the $\mathcal{X}$ token from a $\mathcal{Y} \to \mathcal{X}$ trade immediately on the external market and buys the required $\mathcal{X}$ token for a $\mathcal{X} \to \mathcal{Y}$ trade as needed. In RediSwap, a correct mental model is to manage arbitrageur i's authorized capital in her account, calculate the net balance change after the entire mechanism is executed, and then determine the total profit all at once. It is worth noting that these two mental models are not equivalent when $v_{i,0} \neq v_{i,1}$, and under this more realistic model the modified mechanism is not incentive-compatible.

Budget Constraints. The mechanism assumes that arbitrageurs provide sufficient authorization, that is, they are willing and able to buy or sell arbitrary sizes at their quoted valuation. In reality, however, individual arbitrageurs may face a limited budget, restricting their ability to fully exploit the allocated MEV opportunities. To incorporate the budget constraints, we may ask arbitrageurs to report their available budget as well. However, ensuring that the mechanism remains truthful and Sybil-proof in this setting may require a compromise on efficiency; see for example the impossibility theorems [57, 58].

User's Incentive. While our paper primarily focuses on arbitrageurs' incentives, users' incentives can be an interesting consideration. In a fully random scenario — where the user has no information — they naturally just submit their best guess as the limit price. On the other hand, if a user had complete knowledge of all arbitrageurs' true beliefs, they could manipulate the outcome by strategically setting the limit price. This scenario, however, is unrealistic in practice. Between these two extremes, a more plausible scenario is one in which users' beliefs are drawn from a distribution characteristic of noise traders. To design a mechanism that is incentive-compatible for both users and arbitrageurs under this assumption, insights from the literature on double auctions may be valuable [59, 60, 61].

Implementation Considerations. This paper focuses on the mechanism design problem, and we leave its concrete implementation for future work, for which we do not foresee significant challenges. RediSwap consists of a CFMM pool (a smart contract) and an MEV-redistribution mechanism. The main implementation task is to protect the confidentiality of arbitrageurs' reports[4]. Since most smart contract platforms do not offer confidentiality, the mechanism needs to run in an off-chain environment. This also allows off-loading computation burdens to save on gas costs.

Several tools are available to implement the design. The most straightforward option is to use hardware-based Trusted Execution Environments (TEEs), such as Intel SGX [62] and TDX [63], which are readily available from cloud computing platforms and achieve near-native performance. The high-level workflow is to run the mechanism in a TEE, which accepts (encrypted) inputs from users and arbitrageurs, computes the bundles, and releases the bundle at the appropriate time.

[4]For instance, if the reports $\mathbf{q}$ of all arbitrageurs are public, the user of an $(\mathcal{X} \to \mathcal{Y}, \delta_{\mathcal{X}}^{in}, \delta_{\mathcal{Y}}^{out})$ transaction can strategically set $\delta_{\mathcal{Y}}^{out}/\delta_{\mathcal{X}}^{in} = \max_{i\in[n]} q_i$, in order to get better overall execution. Protecting the confidentiality of arbitrageurs' reports can prevent such manipulation.

An alternative implementation is to use a combination of fully homomorphic encryption and zero-knowledge proofs; because sensitive information only needs to remain private for a short period (e.g., minutes), a lower security level may be used to improve performance.

References

[1] G. Angeris and T. Chitra, "Improved Price Oracles: Constant Function Market Makers," in *Proceedings of the 2nd ACM Conference on Advances in Financial Technologies (AFT '20)*, 2020, pp. 80–91.

[2] P. Daian, S. Goldfeder, T. Kell, Y. Li, X. Zhao, I. Bentov, L. Breidenbach, and A. Juels, "Flash Boys 2.0: Frontrunning in Decentralized Exchanges, Miner Extractable Value, and Consensus Instability," in *2020 IEEE symposium on security and privacy (SP '20)*. IEEE, 2020, pp. 910–927.

[3] K. Qin, L. Zhou, and A. Gervais, "Quantifying Blockchain Extractable Value: How dark is the forest?" in *2022 IEEE Symposium on Security and Privacy (SP '22)*. IEEE, 2022, pp. 198–214.

[4] C. F. Torres, R. Camino *et al.*, "Frontrunner Jones and the Raiders of the Dark Forest: An Empirical Study of Frontrunning on the Ethereum Blockchain," in *30th USENIX Security Symposium (USENIX Security '21)*, 2021, pp. 1343–1359.

[5] L. Zhou, K. Qin, C. F. Torres, D. V. Le, and A. Gervais, "High-Frequency Trading on Decentralized On-Chain Exchanges," in *2021 IEEE Symposium on Security and Privacy (SP '21)*. IEEE, 2021, pp. 428–445.

[6] J. Milionis, C. C. Moallemi, T. Roughgarden, and A. L. Zhang, "Automated Market Making and Loss-Versus-Rebalancing," *arXiv preprint arXiv:2208.06046*, 2022.

[7] Flashbots, "MEV-Share," https://docs.flashbots.net/flashbots-mev-share/introduction, 2025, accessed: 2025-02-08.

[8] MEV Blocker, "MEV Blocker," https://mevblocker.io/, 2023, accessed: 2024-10-08.

[9] CoW DAO, "CoW Protocol Documentation," https://docs.cow.fi/cow-protocol, 2022, accessed: 2024-10-05.

[10] C. Protocol, "MEV Blocker," https://dune.com/cowprotocol/mev-blocker, 2023, accessed: 2024-10-10.

[11] Uniswap, "UniswapX Overview," https://docs.uniswap.org/contracts/uniswapx/overview, 2023, accessed: 2024-10-05.

[12] A. Adams, C. Moallemi, S. Reynolds, and D. Robinson, "am-AMM: An Auction-Managed Automated Market Maker," *arXiv preprint arXiv:2403.03367*, 2024.

[13] Josojo, "MEV capturing AMM (McAMM)," 2022, accessed: 2024-10-08. [Online]. Available: https://ethresear.ch/t/mev-capturing-amm-mcamm/13336

[14] R. Fritsch and A. Canidio, "Measuring Arbitrage Losses and Profitability of AMM Liquidity," in *Companion Proceedings of the ACM on Web Conference (WWW '24)*, 2024, pp. 1761–1767.

[15] J. Milionis, C. C. Moallemi, and T. Roughgarden, "The Effect of Trading Fees on Arbitrage Profits in Automated Market Makers," in *International Conference on Financial Cryptography and Data Security (FC '23)*. Springer, 2023, pp. 262–265.

[16] L. Heimbach, V. Pahari, and E. Schertenleib, "Non-Atomic Arbitrage in Decentralized Finance," in *2024 IEEE Symposium on Security and Privacy (SP '24)*. IEEE Computer Society, 2024, pp. 224–224.

[17] Ethereum Foundation, "The Merge — ethereum.org," 2024, accessed: 2024-10-10. [Online]. Available: https://ethereum.org/en/roadmap/merge/

[18] Sorella, "Sorella dashboard," 2024, accessed: 2024-10-07. [Online]. Available: https://sorellalabs.xyz/dashboard

[19] M. Yokoo, Y. Sakurai, and S. Matsubara, "The effect of false-name bids in combinatorial auctions: New fraud in internet auctions," *Games and Economic Behavior*, vol. 46, no. 1, pp. 174–188, 2004.

[20] Y. Cheng, X. Deng, Y. Li, and X. Yan, "Tight incentive analysis of sybil attacks against the market equilibrium of resource exchange over general networks," *Games Econ. Behav.*, vol. 148, pp. 566–610, 2024.

[21] Y. Gafni, R. Lavi, and M. Tennenholtz, "Worst-case bounds on power vs. proportion in weighted voting games with an application to false-name manipulation," *J. Artif. Intell. Res.*, vol. 72, pp. 99–135, 2021.

[22] G. Angeris, A. Evans, T. Chitra, and S. Boyd, "Optimal Routing for Constant Function Market Makers," in *Proceedings of the 23rd ACM Conference on Economics and Computation (EC '22)*, 2022, pp. 115–128.

[23] D. Engel and M. Herlihy, "Composing Networks of Automated Market Makers," in *Proceedings of the 3rd ACM Conference on Advances in Financial Technologies (AFT '23)*, 2021, pp. 15–28.

[24] L. Zhou, K. Qin, and A. Gervais, "A2MM: Mitigating Frontrunning, Transaction Reordering and Consensus Instability in Decentralized Exchanges," *arXiv preprint arXiv:2106.07371*, 2021.

[25] G. Ramseyer, A. Goel, and D. Mazières, "SPEEDEX: A Scalable, Parallelizable, and Economically Efficient Decentralized EXchange," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI '23)*, 2023, pp. 849–875.

[26] M. Zhang, Y. Li, X. Sun, E. Chen, and X. Chen, "Computation of Optimal MEV in Decentralized Exchanges," Working paper-https://mengqian-zhang.github.io/papers/batch.pdf, Tech. Rep., 2024.

[27] A. Canidio and R. Fritsch, "Batching Trades on Automated Market Makers," in *5th Conference on Advances in Financial Technologies (AFT '23)*. Schloss-Dagstuhl-Leibniz Zentrum für Informatik, 2023.

[28] G. Ramseyer, M. Goyal, A. Goel, and D. Mazières, "Augmenting Batch Exchanges with Constant Function Market Makers," *arXiv preprint arXiv:2210.04929*, 2022.

[29] M. V. Xavier Ferreira and D. C. Parkes, "Credible Decentralized Exchange Design via Verifiable Sequencing Rules," in *Proceedings of the 55th Annual ACM Symposium on Theory of Computing (STOC '23)*, 2023, pp. 723–736.

[30] T. Chan, K. Wu, and E. Shi, "Mechanism Design for Automated Market Makers," *arXiv preprint arXiv:2402.09357*, 2024.

[31] S. Wadhwa, L. Zanolini, F. D'Amato, A. Asgaonkar, C. Fang, F. Zhang, and K. Nayak, "Data Independent Order Policy Enforcement: Limitations and Solutions," *Cryptology ePrint Archive*, 2023.

[32] C. McMenamin and V. Daza, "An AMM minimizing user-level extractable value and loss-versus-rebalancing," *arXiv preprint arXiv:2301.13599*, 2023.

[33] Flashbots, "Flashbots Protect Overview," https://docs.flashbots.net/flashbots-protect/overview, 2024, accessed: 2024-10-20.

[34] M. Kelkar, F. Zhang, S. Goldfeder, and A. Juels, "Order-Fairness for Byzantine Consensus," in *40th Annual International Cryptology Conference, (CRYPTO '20)*. Springer, 2020, pp. 451–480.

[35] M. Kelkar, S. Deb, S. Long, A. Juels, and S. Kannan, "Themis: Fast, Strong Order-Fairness in Byzantine Consensus," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS '23)*, 2023, pp. 475–489.

[36] Y. Zhang, S. Setty, Q. Chen, L. Zhou, and L. Alvisi, "Byzantine Ordered Consensus without Byzantine Oligarchy," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI '20)*, 2020, pp. 633–649.

[37] C. Cachin, J. Mićić, N. Steinhauer, and L. Zanolini, "Quick Order Fairness," in *International Conference on Financial Cryptography and Data Security (FC '22)*. Springer, 2022, pp. 316–333.

[38] S. Yang, F. Zhang, K. Huang, X. Chen, Y. Yang, and F. Zhu, "SoK: MEV Countermeasures: Theory and Practice," *arXiv preprint arXiv:2212.05111*, 2022.

[39] T. Roughgarden, "Transaction Fee Mechanism Design," in *Proceedings of the 22nd ACM Conference on Economics and Computation (EC '21)*, P. Biró, S. Chawla, and F. Echenique, Eds. ACM, 2021, p. 792. [Online]. Available: https://doi.org/10.1145/3465456.3467591

[40] M. Bahrani, P. Garimidi, and T. Roughgarden, "Transaction Fee Mechanism Design in a Post-MEV World," in *6th Conference on Advances in Financial Technologies (AFT '24), September 23-25, 2024, Vienna, Austria*, ser. LIPIcs, R. Böhme and L. Kiffer, Eds., vol. 316. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024, pp. 29:1–29:24. [Online]. Available: https://doi.org/10.4230/LIPIcs.AFT.2024.29

[41] H. Chung, T. Roughgarden, and E. Shi, "Collusion-resilience in transaction fee mechanism design," in *Proceedings of the 25th ACM Conference on Economics and Computation*, 2024, pp. 1045–1073.

[42] Y. Gafni and A. Yaish, "Barriers to collusion-resistant transaction fee mechanisms," in *Proceedings of the 25th ACM Conference on Economics and Computation*, 2024, pp. 1074–1096.

[43] A. Ganesh, C. Thomas, and S. M. Weinberg, "Revisiting the Primitives of Transaction Fee Mechanism Design," in *EC*. ACM, 2024, p. 703.

[44] J. Lenzi, "An Efficient and Sybil Attack Resistant Voting Mechanism," *arXiv preprint arXiv:2407.01844*, 2024.

[45] W. Wang, L. Zhou, A. Yaish, F. Zhang, B. Fisch, and B. Livshits, "Mechanism Design for ZK-Rollup Prover Markets," *arXiv preprint arXiv:2404.06495*, 2024.

[46] B. Mazorra and N. Della Penna, "Towards Optimal Prior-Free Permissionless Rebate Mechanisms, with applications to Automated Market Makers & Combinatorial Order-flow Auctions," *arXiv preprint arXiv:2306.17024*, 2023.

[47] J. Milionis, C. C. Moallemi, and T. Roughgarden, "A Myersonian Framework for Optimal Liquidity Provision in Automated Market Makers," in *15th Innovations in Theoretical Computer Science Conference, ITCS 2024, January 30 to February 2, 2024, Berkeley, CA, USA*, ser. LIPIcs, V. Guruswami, Ed., vol. 287. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024, pp. 81:1–81:19. [Online]. Available: https://doi.org/10.4230/LIPIcs.ITCS.2024.81

[48] M. Goyal, G. Ramseyer, A. Goel, and D. Mazières, "Finding the Right Curve: Optimal Design of Constant Function Market Makers," in *Proceedings of the 24th ACM Conference on Economics and Computation, EC 2023, London, United Kingdom, July 9-12, 2023*, K. Leyton-Brown, J. D. Hartline, and L. Samuelson, Eds. ACM, 2023, pp. 783–812. [Online]. Available: https://doi.org/10.1145/3580507.3597688

[49] U. Docs. (2022) Permit2 Overview. [Online]. Available: https://docs.uniswap.org/contracts/permit2/overview

[50] Binance, "ETH Price," 2024, accessed: 2024-10-05. [Online]. Available: https://www.binance.com/en/price/ethereum

[51] Binance, "Binance public data," 2025, accessed: 2025-02-05. [Online]. Available: https://github.com/binance/binance-public-data

[52] D. Analytics, "DEX Trades Dataset," https://dexanalytics.org/schemas/dex-trades, 2024, accessed: 2024-10-05.

[53] Dune Analytics. (2024) Dune. [Online]. Available: https://dune.com/

[54] W. Feller, *An introduction to probability theory and its applications, Volume 2*. John Wiley & Sons, 1991, vol. 81.

[55] B. C. Arnold, "Pareto distribution," *Wiley StatsRef: Statistics Reference Online*, pp. 1–10, 2014.

[56] Etherscan, "Etherscan DEX," https://etherscan.io/dex, 2024, accessed: 2024-10-17.

[57] C. Borgs, J. T. Chayes, N. Immorlica, M. Mahdian, and A. Saberi, "Multi-unit auctions with budget-constrained bidders," in *EC*. ACM, 2005, pp. 44–51.

[58] S. Dobzinski, R. Lavi, and N. Nisan, "Multi-unit Auctions with Budget Limits," in *FOCS*. IEEE Computer Society, 2008, pp. 260–269.

[59] R. P. McAfee, "The gains from trade under fixed price mechanisms," *Applied economics research bulletin*, vol. 1, no. 1, pp. 1–10, 2008.

[60] Y. Deng, J. Mao, B. Sivan, and K. Wang, "Approximately efficient bilateral trade," in *STOC*. ACM, 2022, pp. 718–721.

[61] J. Brustle, Y. Cai, F. Wu, and M. Zhao, "Approximating gains from trade in two-sided markets via simple mechanisms," in *Proceedings of the 2017 ACM Conference on Economics and Computation*, 2017, pp. 589–590.

[62] V. Costan, "Intel SGX explained," *IACR Cryptol, EPrint Arch*, 2016.

[63] Intel, "Intel Trust Domain Extensions (TDX) Documentation," https://www.intel.com/content/www/us/en/developer/tools/trust-domain-extensions/documentation.html, 2023, accessed: 2024-10-23.

[64] C. Protocol, "CowSwap Dashboard," https://dune.com/cowprotocol/cowswap, 2024, accessed: 2024-10-08.

A Analysis of the orders in UniswapX and CoWSwap

In this section, we analyze how the users' orders are fulfilled on UniswapX and CoWSwap. In particular, we are interested in the following questions:

1. How many orders are filled through direct exchanges between users and solvers?
2. How many orders are filled through batch auctions?

The first question examines the percentage of orders directly filled by solvers' assets rather than liquidity pools in DEXs. The second question evaluates the effectiveness of the batch auction process, where orders can be fulfilled together by a solver as a group, known as a *batch*. If a batch contains only one order, it suggests that the batch auction's effectiveness is lower than expected, as there is no counterpart order within the batch.

The answers to these questions reveal how solvers fulfill orders—whether they simply use their own assets or aggregate orders in a more complex manner.

Orders collection. We collect 663,831 orders on CoWSwap from the DEX Analytics Platform [52] during the period from September 1, 2023, to August 31, 2024. We cross-check these CoWSwap orders with records on Dune [64] and find that the number of orders matched. Similarly, we collect 726,789 orders on UniswapX from the DEX Analytics Platform within the same date range and cross-checked them against Dune records. The results from both sources matched, indicating the accuracy of our collected data.

The number of orders filled through direct exchanges. An order can be filled through various solutions, such as existing liquidity pools like Uniswap v2/v3, or through direct exchange between the solver and the user. Since there is no public dataset that directly indicates how an order in UniswapX or CoWSwap is filled, to answer the first question, we need to differentiate the solutions by which an order is filled. A key observation in answering this question is that if an order is directly filled through token exchanges between a filler and a user, *the process does not involve any liquidity pool*.

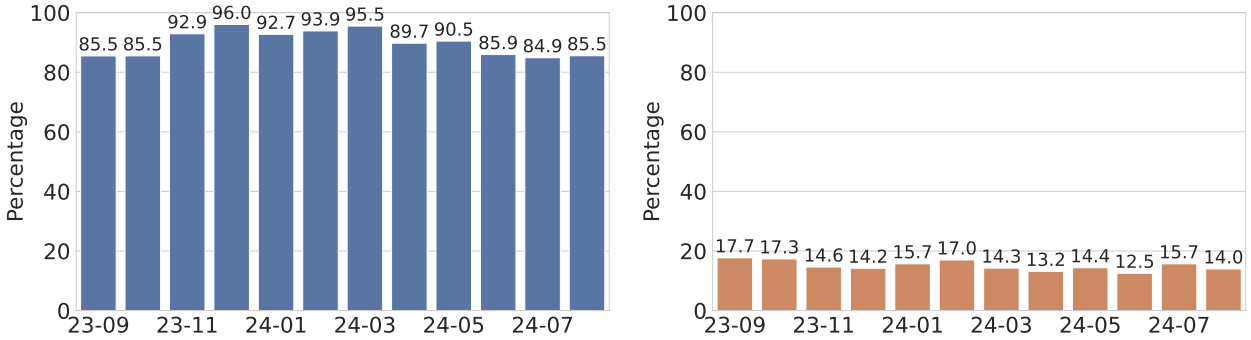
Based on this observation, we use a heuristic to determine whether an order is directly filled through a filler-user token exchange. If a transaction includes only a user order, and this order is filled without involving any liquidity pool, we infer that it must be filled by direct exchange.

We apply this heuristic to the historical orders we collected, and the results are shown in Figure 5. A surprising finding is that over 84% of the orders on UniswapX were filled through direct token exchange between solvers and users, suggesting that this solution is widely used by active solvers on UniswapX. In comparison, a smaller percentage of orders on CoWSwap were filled through direct exchange—the percentage ranges from 12.5% to 17.7%.

One possible explanation for the difference between these two protocols is that different sets of solvers are active in UniswapX and CoWSwap, respectively. For example, an active solver[5] on UniswapX did not participate in CoWSwap.

The number of orders per batch. When collecting historical orders on UniswapX and CoWSwap, we also recorded the hash of the transaction in which each order was filled and the

[5] 0xfbeedcfe378866dab6abbafd8b2986f5c1768737

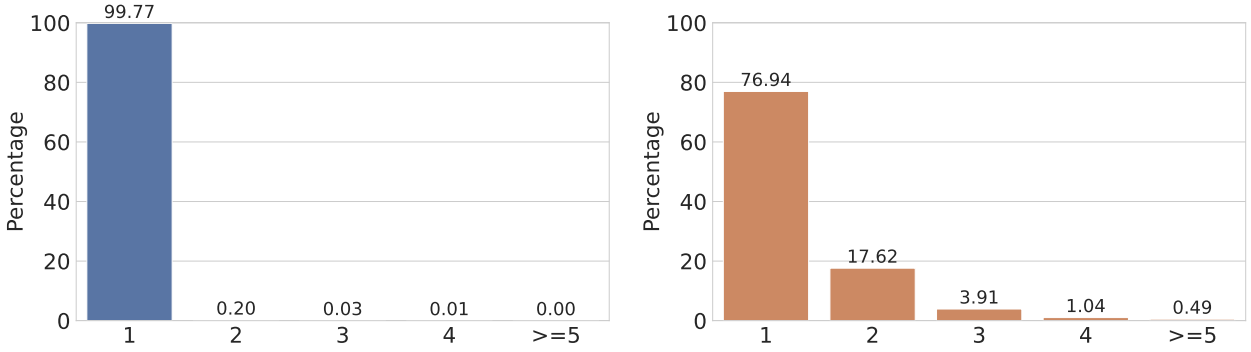


(a) UniswapX (b) CoWSwap

Figure 5: Percentage of orders filled by direct exchange between users and solvers.

solver who fulfilled it. Note that a batch refers to a group of orders filled by a solver within a single transaction. Using the collected data, we can compute the number of orders filled within each batch. The distribution of the number of orders per batch is shown in Figure 6.

An interesting observation from Figure 6a is that over 99% of batches on UniswapX contained only a single filled order. In contrast, as shown in Figure 6b, 17.6% of batches on CoWSwap contained two filled orders, while fewer than 6% of batches contained more than three orders, and fewer than 0.5% included more than five orders.



(a) UniswapX (b) CoWSwap

Figure 6: Distribution of the number of orders per batch.

B Missing Proofs

B.1 Proof of Theorem 1 and Theorem 2

We prove these two theorems together in the following.

Proof. Fix an arbitrary arbitrageur i, its true belief v_i, the reports $\mathbf{q}_{-i}$ of all other arbitrageurs, and their Sybil transactions S_{-i}. Recalling definition 6 and definition 7, we only

need to focus on the case where arbitrageur i submits no Sybil transaction, i.e., $S_i = \emptyset$. Consequently, the formula (1a) equals 0, and arbitrageur i's utility can be rewritten as the cumulative profit from $|M|+1$ simultaneous auctions. Then, it suffices to show that in each auction, truthful reporting is a dominant strategy for every arbitrageur, and truthful arbitrageurs always obtain nonnegative utility. Based on the MEV opportunity being auctioned, there are two cases: a transaction or the initial state.

Case 1: an arbitrary transaction TX_j. Suppose arbitrageur i reports q_i. The mechanism computes $\Phi_i(\mathsf{TX}_j) = \Delta x_j \cdot q_i + \Delta y_j$ as her bid. Consider the scenario that i wins and the mechanism constructs a "sandwich" for her. According to the formula (1b), i's revenue from this is

$$\underbrace{(x_0 - x_j^\ell) \cdot v_i^* + (y_0 - y_j^\ell)}_{\text{Front-running: } (x_0, y_0) \to (x_j^\ell, y_j^\ell)} + \underbrace{(x_j^\ell + \Delta x_j - x_0) \cdot v_i^* + (y_j^\ell + \Delta y_j - y_0)}_{\text{Back-running: } (x_j^\ell + \Delta x_j, y_j^\ell + \Delta y_j) \to (x_0, y_0)} = \Delta x_j \cdot v_i^* + \Delta y_j, \quad (9)$$

which is fixed (independent of q_i). If i's "true valuation" $\Delta x_j \cdot v_i^* + \Delta y_j \geq \max_{k \neq i} \Phi_k(\mathsf{TX}_j)$, the maximum utility that i can obtain is $\max\{0, \Delta x_j \cdot v_i^* + \Delta y_j - \max_{k \neq i} V_k(\mathsf{TX}_j)\}$, and is achieved by reporting truthfully (and winning). Otherwise, i.e., $\Delta x_j \cdot v_i^* + \Delta y_j < \max_{k \neq i} \Phi_k(\mathsf{TX}_j)$, the maximum utility that i can obtain is 0 and i achieves by reporting truthfully (and losing).

In both cases, arbitrageur i obtains nonnegative utility.

Case 2: the initial state. According to arbitrageur i's report q_i, the mechanism computes $\phi(s_0, q_i) = (x_0 \cdot q_i + y_0) - (\hat{x}_i \cdot q_i + \hat{y}_i)$ as her bid. Consider the scenario that arbitrageur i wins and the mechanism inserts a rebalancing arbitrage transaction for her that stops at the state $(\hat{x}_i, \hat{y}_i)$ corresponding to q_i. According to the formula (1b), i's revenue is

$$\begin{aligned} h(q_i) \coloneqq \overbrace{(x_0 - \hat{x}_i) \cdot v_i^* + (y_0 - \hat{y}_i)}^{\text{Rebalancing: } (x_0, y_0) \to (\hat{x}_i, \hat{y}_i)} &= (x_0 \cdot v_i^* + y_0) - (\hat{x}_i \cdot v_i^* + \hat{y}_i) \\ &= (x_0 \cdot v_i^* + y_0) - ((x_i^* \cdot v_i^* + y_i^*) + (\hat{x}_i \cdot v_i^* + \hat{y}_i) - (x_i^* \cdot v_i^* + y_i^*)) \\ &= (x_0 \cdot v_i^* + y_0) - (x_i^* \cdot v_i^* + y_i^*) - \phi(\hat{s}_i, v_i^*) \\ &= \phi(s_0, v_i^*) - \phi(\hat{s}_i, v_i^*) \leq \phi(s_0, v_i^*). \end{aligned}$$

Namely, i's revenue $h(q_i)$ is maximized when i reports truthfully. The last "$\leq$" is because $\phi(\hat{s}_i, v_i^*) \geq 0$ by Claim 1 and "$=$" is reached when $\hat{s}_i = s^*$.

The remaining analysis is similar. If arbitrageur i's "true valuation" $h(v_i^*) \geq \max_{k \neq i} \phi_k(s_0)$, the maximum utility that i can obtain is $\max\{0, h(v_i^*) - \max_{k \neq i} \phi_k(s_0)\} = \phi(s_0, v_i^*)$, and is achieved by reporting truthfully (and winning). Otherwise, i.e., $h(v_i^*) < \max_{k \neq i} \phi_k(s_0)$, the maximum utility that i can obtain is $\max\{0, h(v_i^*) - \max_{k \neq i} \phi_k(s_0)\} = 0$ and i achieves by reporting truthfully (and losing). In both cases, arbitrageur i obtains nonnegative utility.

This finishes the proof. □

B.2 Proof of Theorem 3

Proof of Theorem 3. Fix an arbitrary arbitrageur i and the reports $\mathbf{q}$ of all arbitrageurs. Whether a transaction $\mathsf{TX}_j \in M$ can be included in the bundle and thus executed only

depends on $\max_{k\in[n]} \Delta x_j \cdot q_k + \Delta y_j$, where $(\Delta x_j, \Delta y_j)$ is determined by TX_j itself (see eq. (6)). Suppose the maximal value is Φ_w achieved by arbitrageur $w \in [n]$. If $\Phi_w \geq 0$, TX_j will be deterministically executed at its limit state, causing a deterministic change in the user's account balance, and receive a refund $\max_{k\neq w} \max\{0, \Delta x_j \cdot q_k + \Delta y_j\}$. As a result, the number of tokens that the user of TX_j owns after executing through our mechanism is only determined by the transaction itself and $\mathbf{q}$, which are fixed and independent of arbitrageur i's strategy S_i. This concludes the proof. □

B.3 Proof of Observation 1

Proof of Observation 1. Fix arbitrary inputs. Without loss of generality, suppose $q_1 \geq q_2 \geq \cdots \geq q_n$. For any transaction $\mathsf{TX}_j \in M$, let's denote its winner by w_j.

If $\mathsf{TX}_j = (\mathcal{X} \to \mathcal{Y}, \delta^{in}_{\mathcal{X}}, \delta^{out}_{\mathcal{Y}})$, $w_j = \arg\max_{i\in[n]} V_i(\mathsf{TX}_j) = \arg\max_{i\in[n]} \max\left\{0, \delta^{in}_{\mathcal{X}} \cdot q_i - \delta^{out}_{\mathcal{Y}}\right\}$. It is straightforward to see that the winner is always arbitrageur 1. If the limit price $\delta^{out}_{\mathcal{Y}}/\delta^{in}_{\mathcal{X}} > q_1$, TX_j will be ignored according to Algorithm 1. Otherwise, the user of TX_j pays $\delta^{in}_{\mathcal{X}}$ units of $\mathcal{X}$ and receives $\delta^{out}_{\mathcal{Y}}$ (when $n = 1$) or $\delta^{out}_{\mathcal{Y}} + \max\left\{0, \delta^{in}_{\mathcal{X}} \cdot q_2 - \delta^{out}_{\mathcal{Y}}\right\}$ (when $n \geq 2$) units of $\mathcal{Y}$ token, where $\max\left\{0, \delta^{in}_{\mathcal{X}} \cdot q_2 - \delta^{out}_{\mathcal{Y}}\right\}$ is the refund. By taking the refund into account, the *overall* execution price is either $\delta^{out}_{\mathcal{Y}}/\delta^{in}_{\mathcal{X}}$ (when $n = 1$) or

$$\frac{\delta^{out}_{\mathcal{Y}} + \max\left\{0, \delta^{in}_{\mathcal{X}} \cdot q_2 - \delta^{out}_{\mathcal{Y}}\right\}}{\delta^{in}_{\mathcal{X}}} = \max\left\{q_2, \delta^{out}_{\mathcal{Y}}/\delta^{in}_{\mathcal{X}}\right\}.$$

Similarly, if $\mathsf{TX}_j = (\mathcal{Y} \to \mathcal{X}, \delta^{in}_{\mathcal{Y}}, \delta^{out}_{\mathcal{X}})$, $w_j = \arg\max_{i\in[n]} \max\left\{0, -\delta^{out}_{\mathcal{X}} \cdot q_i + \delta^{in}_{\mathcal{Y}}\right\}$. So arbitrageur n, whose report is the minimum, always wins. When $\delta^{in}_{\mathcal{Y}}/\delta^{out}_{\mathcal{X}} \geq q_n$, the user of TX_j pays $\delta^{in}_{\mathcal{Y}}$ units of $\mathcal{Y}$ token while receives $\delta^{out}_{\mathcal{X}}$ units of $\mathcal{X}$ token and $\max\left\{0, -\delta^{out}_{\mathcal{X}} \cdot q_{n-1} + \delta^{in}_{\mathcal{Y}}\right\}$ units of $\mathcal{Y}$ token as refunds when $n \geq 2$. As a result, the *overall* execution price is either $\delta^{in}_{\mathcal{Y}}/\delta^{out}_{\mathcal{X}}$ (when $n = 1$) or

$$\frac{\delta^{in}_{\mathcal{Y}} - \max\left\{0, -\delta^{out}_{\mathcal{X}} \cdot q_{n-1} + \delta^{in}_{\mathcal{Y}}\right\}}{\delta^{out}_{\mathcal{X}}} = \min\left\{q_{n-1}, \delta^{in}_{\mathcal{Y}}/\delta^{out}_{\mathcal{X}}\right\}.$$

□

B.4 Proof of Theorem 4

Proof of Theorem 4. Our proof strategy below follows from two steps: We first give a comprehensive analysis of arbitrageur i's profit with arbitrary strategy given everyone's belief v^*_i. The analysis will provide the intuition of our definition of the Sybil strategy. We then formally define the Sybil strategy $S_i(v^*_i, b^{\mathcal{X}}_i, b^{\mathcal{Y}}_i, \mathcal{D})$ and show that $(v^*_i, S_i(v^*_i, b^{\mathcal{X}}_i, b^{\mathcal{Y}}_i, \mathcal{D}))$ is a best strategy of arbitrageur i when everyone else follows this strategy and their belief v^*_k is drawn from $\mathcal{D}_k$.

First step. Fix an arbitrageur i, its true belief v^*_i, its budget $(b^{\mathcal{X}}_i, b^{\mathcal{Y}}_i)$, and the strategies of all other arbitrageurs $\left\{(v^*_k, S_k(v^*_k, b^{\mathcal{X}}_k, b^{\mathcal{Y}}_k, \mathcal{D}))\right\}_{k\neq i}$. Here let's say $S_k(v^*_k, b^{\mathcal{X}}_k, b^{\mathcal{Y}}_k, \mathcal{D})$ is some

abstract Sybil strategy and it doesn't affect the analysis in the first step. We use S_k as a shortening of $S_k(v_k^*, b_k^{\mathcal{X}}, b_k^{\mathcal{Y}}, \mathcal{D})$ for every $k \in [n]$ for simplicity of notations.

Given an arbitrary arbitrageur i's strategy (q_i, S_i'), i's utility $u(S_i', q_i; S_{-i}, \mathbf{v}_{-i}^*)$ can be calculated by enumerating all sub-bundles in the output bundle of our mechanism. Our analysis and proof will be based on analyzing the profit of two different groups of the sub-bundles.

The proof of Theorem 1 implies that for every sub-bundle for which the middle transaction $e \in R \cup S_{-i} \cup \{s_0\}$, we have the profit from reporting q_i is no more than the profit from reporting v_i^*. Thus, we will be mainly focusing on the arbitrageur i's profit of the sub-bundle of every $e \in S_i'$ when i reports q_i.

Let's consider any $e \in S_i'$ such that $e = \mathsf{TX} = (\mathcal{X} \to \mathcal{Y}, \delta_{\mathcal{X}}^{in}, \delta_{\mathcal{Y}}^{out})$. The other case will be similar. Note that if TX is sandwiched by i itself (which means $q_i \geq \max_{k \neq i}\{v_k^*\}$), then the profit of i is 0; if TX is sandwiched by someone else, denoted by w, then arbitrageur i's utility from this sub-bundle is

$$H(q_i, \delta_{\mathcal{Y}}^{out}) = \underbrace{\delta_{\mathcal{Y}}^{out} - \delta_{\mathcal{X}}^{in} \cdot v_i^*}_{\text{Execution of } \mathsf{TX}} + \underbrace{\max\left(0, \delta_{\mathcal{X}}^{in} \cdot q_i - \delta_{\mathcal{Y}}^{out}, \max_{k \neq i,w}\{\delta_{\mathcal{X}}^{in} \cdot v_k^* - \delta_{\mathcal{Y}}^{out}\}\right)}_{\text{Refund to } \mathsf{TX}}. \tag{10}$$

To analyze the profit formula above, note that $H(\cdot, \cdot)$ is an increasing function of both parameters. However, there are also two upper bounds of these parameters based on the fact that this sub-bundle is won and sandwiched by $w \neq i$:

- w is the winner of all arbitrageur, which means $q_i \leq v_w^*$;
- the profit of w is no less than 0, which means $\delta_{\mathcal{Y}}^{out} \leq \delta_{\mathcal{X}}^{in} \cdot v_w^*$.

We need the further property of $H(\cdot, \cdot)$, stated below:

Claim 2. *For any t such that $t \leq v_w^*$, we have $H(t, \delta_{\mathcal{X}}^{in} \cdot t) = H(q_i, \delta_{\mathcal{X}}^{in} \cdot t)$ for all $q_i \leq t$.*

Proof. Note that for the parameter regime that we are considering, we have $\delta_{\mathcal{X}}^{in} \cdot q_i - \delta_{\mathcal{X}}^{in} \cdot t \leq 0$. Thus

$$H(q_i, \delta_{\mathcal{X}}^{in} \cdot t) = H(t, \delta_{\mathcal{X}}^{in} \cdot t) = \delta_{\mathcal{X}}^{in} \cdot (t - v_i^*) + \max\left(0, \max_{k \neq i,w}\{\delta_{\mathcal{X}}^{in} \cdot v_k^* - \delta_{\mathcal{Y}}^{out}\}\right).$$

□

The claim above essentially says that, the arbitrageur i's profit from *its own Sybil transactions* is the same for any report $q_i \leq \delta_{\mathcal{Y}}^{out}/\delta_{\mathcal{X}}^{in}$ regardless how small it is. Note that the arbitrageur i also has the profit from users' transactions. Thus this provides a good intuition about what kind of Sybil strategy everyone is using could form a Nash equilibrium. We formalize it in the second step.

Second step. We first specify the Sybil strategy $S_i(v_i^*, b_i^{\mathcal{X}}, b_i^{\mathcal{Y}}, \mathcal{D})$ for every arbitrageur i. Fix an arbitrageur i, it includes two Sybil transactions: $\mathsf{TX} = (\mathcal{X} \to \mathcal{Y}, \delta_{\mathcal{X}}^{in} = b_i^{\mathcal{X}}, \delta_{\mathcal{Y}}^{out} = t_{\mathcal{Y}}^*)$ and $\mathsf{TX} = (\mathcal{Y} \to \mathcal{X}, \delta_{\mathcal{Y}}^{in} = b_i^{\mathcal{Y}}, \delta_{\mathcal{X}}^{out} = t_{\mathcal{X}}^*)$. To specify the $t_{\mathcal{Y}}^*$ above, we recall the profit function of i given q_i and $\delta_{\mathcal{Y}}^{out}$ as parameters:

$$\Gamma(q_i, \delta_{\mathcal{Y}}^{out}, v_{-i}^*)_i = \begin{cases} H(q_i, \delta_{\mathcal{Y}}^{out}) & q_i \le \max_{k \neq i}\{v_k^*\} \text{ and } \delta_{\mathcal{Y}}^{out} \le b_i^{\mathcal{X}} \cdot \max_{k \neq i}\{v_k^*\}; \\ 0 & \text{otherwise.} \end{cases}$$

Importantly, note that in the definition of $\Gamma()$ above, we should use $b_i^{\mathcal{X}}$ as the parameter of $\delta_{\mathcal{X}}^{in}$ when we refer the H function of Equation (10).

Now we are ready to define $t_{\mathcal{Y}}^*$ and $t_{\mathcal{X}}^*$ can be defined similarly.

$$t_{\mathcal{Y}}^* := \arg\max_t \mathbb{E}_{\boldsymbol{v}_{-i}^* \sim \mathcal{D}_{-i}} \left[\Gamma(v_i^*, t, \boldsymbol{v}_{-i}^*)_i\right].$$

Namely, we choose $t_{\mathcal{Y}}^*$ that maximizes the expected profit of *Sybil transactions* given that i *reports* v_i^* *truthfully*. Note that by Theorem 1, reporting truthfully can maximize arbitrageur i's profit of transactions that are not i's Sybil transactions. Thus, we only need to show that arbitrageur i's profit of *its Sybil transactions* is maximized when i uses our specified strategy, comparing with arbitrary report q_i and set of Sybil transactions S_i'. We consider the case where S_i' only contains one Sybil transaction of $(\mathcal{X} \to \mathcal{Y}, \delta_{\mathcal{X}}^{in} = b_i^{\mathcal{X}}, \delta_{\mathcal{Y}}^{out})$ (and only contains one Sybil transaction of $(\mathcal{Y} \to \mathcal{X}, \delta_{\mathcal{Y}}^{in} = b_i^{\mathcal{Y}}, \delta_{\mathcal{X}}^{out})$). This is without loss of generality because we can easily merge multiple Sybil transactions in the same direction. It is easy to see that $\delta_{\mathcal{Y}}^{out}$ should be at least $b_i^{\mathcal{X}} \cdot v_i^*$.

It remains for us to show that $\mathbb{E}_{\boldsymbol{v}_{-i}^* \sim \mathcal{D}_{-i}} \left[\Gamma(v_i^*, t_{\mathcal{Y}}^*, \boldsymbol{v}_{-i}^*)_i\right] \ge \mathbb{E}_{\boldsymbol{v}_{-i}^* \sim \mathcal{D}_{-i}} \left[\Gamma(q_i, \delta_{\mathcal{Y}}^{out}, \boldsymbol{v}_{-i}^*)_i\right]$. The proof will be purely based on the properties of the H function. Recall that H is increasing for both parameters and Γ is non-zero only if $q_i \le \max_{k \neq i}\{v_k^*\}$ and $\delta_{\mathcal{Y}}^{out} \le b_i^{\mathcal{X}} \cdot \max_{k \neq i}\{v_k^*\}$. Letting $\alpha = \max(q_i, \delta_{\mathcal{Y}}^{out}/b_i^{\mathcal{X}})$, we have that

$$\mathbb{E}_{\boldsymbol{v}_{-i}^* \sim \mathcal{D}_{-i}} \left[\Gamma(\alpha, b_i^{\mathcal{X}} \cdot \alpha, \boldsymbol{v}_{-i}^*)_i\right] \ge \mathbb{E}_{\boldsymbol{v}_{-i}^* \sim \mathcal{D}_{-i}} \left[\Gamma(q_i, \delta_{\mathcal{Y}}^{out}, \boldsymbol{v}_{-i}^*)_i\right].$$

Furthermore, by Claim 2, we know that

$$\mathbb{E}_{\boldsymbol{v}_{-i}^* \sim \mathcal{D}_{-i}} \left[\Gamma(v_i^*, b_i^{\mathcal{X}} \cdot \alpha, \boldsymbol{v}_{-i}^*)_i\right] \ge \mathbb{E}_{\boldsymbol{v}_{-i}^* \sim \mathcal{D}_{-i}} \left[\Gamma(\alpha, b_i^{\mathcal{X}} \cdot \alpha, \boldsymbol{v}_{-i}^*)_i\right].$$

Finally, by the definition of $t_{\mathcal{Y}}^*$, we conclude

$$\mathbb{E}_{\boldsymbol{v}_{-i}^* \sim \mathcal{D}_{-i}} \left[\Gamma(v_i^*, t_{\mathcal{Y}}^*, \boldsymbol{v}_{-i}^*)_i\right] \ge \mathbb{E}_{\boldsymbol{v}_{-i}^* \sim \mathcal{D}_{-i}} \left[\Gamma(v_i^*, b_i^{\mathcal{X}} \cdot \alpha, \boldsymbol{v}_{-i}^*)_i\right].$$

The optimal choice of $t_{\mathcal{X}}^*$ and its analysis is analogous. This concludes the proof. □

C Revised Mechanism for the Two-Belief Setting

C.1 Revised Notations

We revise notations of the arbitrageur's belief, report, and utility function, and the valuation functions of the pool/transaction as follows.

Each arbitrageur i holds a private belief $v_i = (v_{i,0}, v_{i,1})$ about the external price of the risky asset $\mathcal{X}$. Specifically, $v_{i,0}$ denotes the price at which they can sell $\mathcal{X}$ and $v_{i,1}$ the price at which they can buy $\mathcal{X}$ on the external market (e.g., a CEX such as Binance). We use $\mathbf{q}$ to denote the n-sized vector where $q_i = (q_{i,0}, q_{i,1})$ represents arbitrageur i's report on external prices of selling and buying the risky asset $\mathcal{X}$.

Definition 10 (Arbitrageur Utility Function)**.** *For an MEV-redistribution mechanism* $(\mathbf{b}, \mathbf{p}, \mathbf{r})$*, pool's initial state* s_0*, pending transactions* M*, arbitrageurs' report* $\mathbf{q}$*, and generated bundle* B*, the utility of arbitrageur* i *with true belief* v_i *and Sybil transactions* $S_i \subseteq M$ *is*

$$
\begin{aligned}
u\big(S_i, q_i; S_{-i}, \mathbf{q}_{-i}\big) \coloneqq \quad & \sum_{\mathsf{TX}_j \in S_i \cap B} \Bigg[\mathbb{1}_{\{x_j < x_{j-1}\}} \cdot \left((x_{j-1} - x_j) \cdot v_{i,0} + \frac{y_{j-1} - y_j}{1-f} \right) \\
& \qquad + \mathbb{1}_{\{x_j > x_{j-1}\}} \cdot \left(\frac{x_{j-1} - x_j}{1-f} \cdot v_{i,1} + (y_{j-1} - y_j) \right) + r\left(\mathsf{TX}_j\right) \Bigg] && \text{(11a)} \\
+ \quad & \sum_{\mathsf{TX}_j \in A_i} \Big[\mathbb{1}_{\{x_j < x_{j-1}\}} \cdot \left((x_{j-1} - x_j) \cdot v_{i,0} + (y_{j-1} - y_j) \right) \\
& \qquad + \mathbb{1}_{\{x_j > x_{j-1}\}} \cdot \left((x_{j-1} - x_j) \cdot v_{i,1} + (y_{j-1} - y_j) \right) \Big] && \text{(11b)} \\
- \quad & p_i, && \text{(11c)}
\end{aligned}
$$

where $v_{i,0}$ *(*$v_{i,1}$*) is the price at which* i *can sell (buy)* $\mathcal{X}$ *on the external market;* $S_i \cap B$ *is* i*'s Sybil transactions included in the bundle;* $A_i \subseteq B$ *is* i*'s arbitrage transactions inserted by the bundle generation rule* $\mathbf{b}$*;* $f \in [0, 1)$ *is the fraction of swap fees charged by the pool;* (x_{j-1}, y_{j-1}) *and* (x_j, y_j) *are the pool states before and after executing* TX_j*, respectively;* $\mathbb{1}_{\{x_j > x_{j-1}\}}$ *indicates that* TX_j *is an* $\mathcal{X} \to \mathcal{Y}$ *transaction, and* $\mathbb{1}_{\{y_j > y_{j-1}\}}$ *indicates a* $\mathcal{Y} \to \mathcal{X}$ *transaction.*

For any arbitrageur i with the report $q_i = (q_{i,0}, q_{i,1})$, we modify the *pool value* of any pool state $s = (x, y)$ to be

$$
\phi(s, q_i) \coloneqq \begin{cases} (x \cdot q_{i,0} + y) - (\hat{x}_{i,0} \cdot q_{i,0} + \hat{y}_{i,0}), & \text{if } x \geq \hat{x}_{i,0}; \\ (x \cdot q_{i,1} + y) - (\hat{x}_{i,1} \cdot q_{i,1} + \hat{y}_{i,1}), & \text{if } x \leq \hat{x}_{i,1}; \\ 0, & \text{otherwise.} \end{cases} \tag{12}
$$

The *potential value* of an arbitrary transaction TX_j to the arbitrageur i with the report q_i is revised as

$$
\Phi(\mathsf{TX}_j, q_i) \coloneqq \begin{cases} \delta^{in}_{\mathcal{X}} \cdot q_{i,0} - \delta^{out}_{\mathcal{Y}}, & \text{when } \mathsf{TX}_j = (\mathcal{X} \to \mathcal{Y}, \delta^{in}_{\mathcal{X}}, \delta^{out}_{\mathcal{Y}}); \\ -\delta^{out}_{\mathcal{X}} \cdot q_{i,1} + \delta^{in}_{\mathcal{Y}}, & \text{when } \mathsf{TX}_j = (\mathcal{Y} \to \mathcal{X}, \delta^{in}_{\mathcal{Y}}, \delta^{out}_{\mathcal{X}}). \end{cases} \tag{13}
$$

Based on $\Phi(\mathsf{TX}_j, q_i)$, the *actual value* of transaction TX_j to arbitrageur i remains

$$
V(\mathsf{TX}_j, q_i) = \max\left\{0, \Phi(\mathsf{TX}_j, q_i)\right\}. \tag{14}
$$

C.2 Revised Mechanism

- **Bundle generation rule:** The revised mechanism constructs the bundle following Algorithm 2, which consists of two parts. The first part (see line 1 - 23) goes over all pending transactions. Each iteration starts with computing the limit state (x_j^ℓ, y_j^ℓ) of transaction $\mathsf{TX}_j \in M$ and the corresponding impact on the pool $(\Delta x_j, \Delta y_j)$. Then, the mechanism selects the winner w_j of this iteration:
  - If $\mathsf{TX}_j = (\mathcal{X} \to \mathcal{Y}, \delta_{\mathcal{X}}^{in}, \delta_{\mathcal{Y}}^{out})$ and $\delta_{\mathcal{Y}}^{out}/\delta_{\mathcal{X}}^{in} \leq \max_{i\in[n]} q_{i,0}$, the winner $w_j = \arg\max_{i\in[n]} q_{i,0}$.
  - If $\mathsf{TX}_j = (\mathcal{Y} \to \mathcal{X}, \delta_{\mathcal{Y}}^{in}, \delta_{\mathcal{X}}^{out})$ and $\delta_{\mathcal{Y}}^{in}/\delta_{\mathcal{X}}^{out} \geq \min_{i\in[n]} q_{i,1}$, the winner $w_j = \arg\min_{i\in[n]} q_{i,1}$.
  - Otherwise, there is no winner, and the mechanism skips this transaction.

  After identifying the winner w_j (if exists), the mechanism assigns TX_j to w_j by constructing a "sandwich" on her behalf. Specifically, the mechanism inserts a front-running transaction from state (x_0, y_0) to (x_j^ℓ, y_j^ℓ) so that TX_j executes at its limit state, followed by a back-running transaction from the post-execution state $(x_j^\ell + \Delta x_j, y_j^\ell + \Delta y_j)$ to the initial state (x_0, y_0).

  The second part of Algorithm 2 (see line 24 - 32) sells the rebalancing arbitrage opportunity. Let $\rho_0 = \left|\frac{\partial F/\partial x}{\partial F/\partial y}(x_0, y_0)\right|$ be the spot price of the initial state $s_0 = (x_0, y_0)$. If $\max_{i\in[n]} q_{i,0} < \rho_0 < \min_{i\in[n]} q_{i,1}$, there is no winner. Otherwise, the mechanism computes the pool value of the initial state to each arbitrageur $i \in [n]$, i.e., $\phi_i(s_0)$, and assigns it to the arbitrageur with the highest value, namely, the winner $w' = \arg\max_{i\in[n]} \phi_i(s_0)$. Specifically, the mechanism adds an arbitrage transaction on behalf of w' to reach the no-arbitrage state in her opinion, which is $(\hat{x}_{w',0}, \hat{y}_{w',0})$ if $x_0 > \hat{x}_{w',0}$, or $(\hat{x}_{w',1}, \hat{y}_{w',1})$ if $x_0 < \hat{x}_{w',1}$ (i.e., the states correspond to w''s report $q_{w',0}$ and $q_{w',1}$, respectively.).

- **Payment rule:** Through the bundle generation, there are at most $|M|+1$ winners (note that an arbitrageur may win multiple times), corresponding to $|M|$ pending transactions and the initial state. The payment rule requires each winner to pay the second highest value in units of the numéraire. Specifically, for each transaction TX_j, the winner w_j pays $\max_{i\neq w_j} V_i(\mathsf{TX}_j)$ and all others pay zero; for the initial state, the winner w' pays $\max_{i\neq w'} \phi_i(s_0)$ while others pay zero. In both cases, if there is no winner, all arbitrageurs pay nothing.

- **Refund rule:** The above payment is refunded to users and liquidity providers, respectively. Specifically, if the winner exists, the owner of transaction TX_j gets $\max_{i\neq w_j} V_i(\mathsf{TX}_j)$ and LPs get $\max_{i\neq w'} \phi_i(s_0)$.

Algorithm 2: Bundle Generation Rule of the Revised Mechanism

Input: An initial state $s_0 = (x_0, y_0)$, a set of pending transactions M, and arbitrageurs' reports $\mathbf{q} = (q_1, \cdots, q_n)$.

Output: A bundle to be executed by CFMM.

for *each* $j \in [1 : |M|]$ **do**
  if $\mathsf{TX}_j = (\mathcal{X} \to \mathcal{Y}, \delta^{in}_{\mathcal{X}}, \delta^{out}_{\mathcal{Y}})$ **then**
    if $\delta^{out}_{\mathcal{Y}}/\delta^{in}_{\mathcal{X}} > \max_{i\in[n]} q_{i,0}$ **then**
      continue.
    (x^ℓ_j, y^ℓ_j) is the limit state satisfying $y^\ell_j - F_y(x^\ell_j + \delta^{in}_{\mathcal{X}}) = \delta^{out}_{\mathcal{Y}}$.
    Let $\Delta x \leftarrow \delta^{in}_{\mathcal{X}}$, $\Delta y \leftarrow -\delta^{out}_{\mathcal{Y}}$.
    Let $w_j \leftarrow \arg\max_{i\in[n]} q_{i,0}$. `// Arbitrageur` w_j `values` TX_j `the most`
  else if $\mathsf{TX}_j = (\mathcal{Y} \to \mathcal{X}, \delta^{in}_{\mathcal{Y}}, \delta^{out}_{\mathcal{X}})$ **then**
    if $\delta^{in}_{\mathcal{Y}}/\delta^{out}_{\mathcal{X}} < \min_{i\in[n]} q_{i,1}$ **then**
      continue.
    (x^ℓ_j, y^ℓ_j) is the limit state satisfying $x^\ell_j - F_x(y^\ell_j + \delta^{in}_{\mathcal{Y}}) = \delta^{out}_{\mathcal{X}}$.
    Let $\Delta x \leftarrow -\delta^{out}_{\mathcal{X}}$, $\Delta y \leftarrow \delta^{in}_{\mathcal{Y}}$.
    Let $w_j \leftarrow \arg\min_{i\in[n]} q_{i,1}$. `// Arbitrageur` w_j `values` TX_j `the most`
  if $x_0 < x^\ell_j$ **then**
    Insert a front-running transaction on behalf of w_j
    $\mathsf{TX} = \big(\mathcal{X} \to \mathcal{Y}, \delta^{in}_{\mathcal{X}} = x^\ell_j - x_0, \delta^{out}_{\mathcal{Y}} = y_0 - y^\ell_j\big)$.
  else if $x_0 > x^\ell_j$ **then**
    Insert a front-running transaction on behalf of w_j
    $\mathsf{TX} = \big(\mathcal{Y} \to \mathcal{X}, \delta^{in}_{\mathcal{Y}} = y^\ell_j - y_0, \delta^{out}_{\mathcal{X}} = x_0 - x^\ell_j\big)$.
  Place the user transaction TX_j.
  Let the current state $(x', y') \leftarrow (x^\ell_j + \Delta x, y^\ell_j + \Delta y)$.
  if $x' < x_0$ **then**
    Insert a back-running transaction on behalf of w_j
    $\mathsf{TX} = \big(\mathcal{X} \to \mathcal{Y}, \delta^{in}_{\mathcal{X}} = x_0 - x', \delta^{out}_{\mathcal{Y}} = y' - y_0\big)$.
  else if $x' > x_0$ **then**
    Insert a back-running transaction on behalf of w_j
    $\mathsf{TX} = \big(\mathcal{Y} \to \mathcal{X}, \delta^{in}_{\mathcal{Y}} = y_0 - y', \delta^{out}_{\mathcal{X}} = x' - x_0\big)$.
Let $\rho_0 \leftarrow$ spot price of the initial state s_0.
if $\rho_0 \le \max_{i\in[n]} q_{i,0}$ ***or*** $\rho_0 \ge \min_{i\in[n]} q_{i,1}$ **then**
  for *each* $i \in [n]$ **do**
    Let $\phi_i \leftarrow$ pool value of the initial state s_0 to arbitrageur i according to eq. (3).
  Let $\phi_{w'} \leftarrow \max_{i\in[n]} \phi_i$. `// Arbitrageur` w' `values` LVR `the most`
  if $x_0 < \hat{x}_{w',1}$ **then**
    Add a transaction on behalf of arbitrageur w'
    $\mathsf{TX} = \big(\mathcal{X} \to \mathcal{Y}, \delta^{in}_{\mathcal{X}} = \hat{x}_{w',1} - x_0, \delta^{out}_{\mathcal{Y}} = y_0 - \hat{y}_{w',1}\big)$.
  else if $x_0 > \hat{x}_{w',0}$ **then**
    Add a transaction on behalf of arbitrageur w'
    $\mathsf{TX} = \big(\mathcal{Y} \to \mathcal{X}, \delta^{in}_{\mathcal{Y}} = \hat{y}_{w',0} - y_0, \delta^{out}_{\mathcal{X}} = x_0 - \hat{x}_{w',0}\big)$.